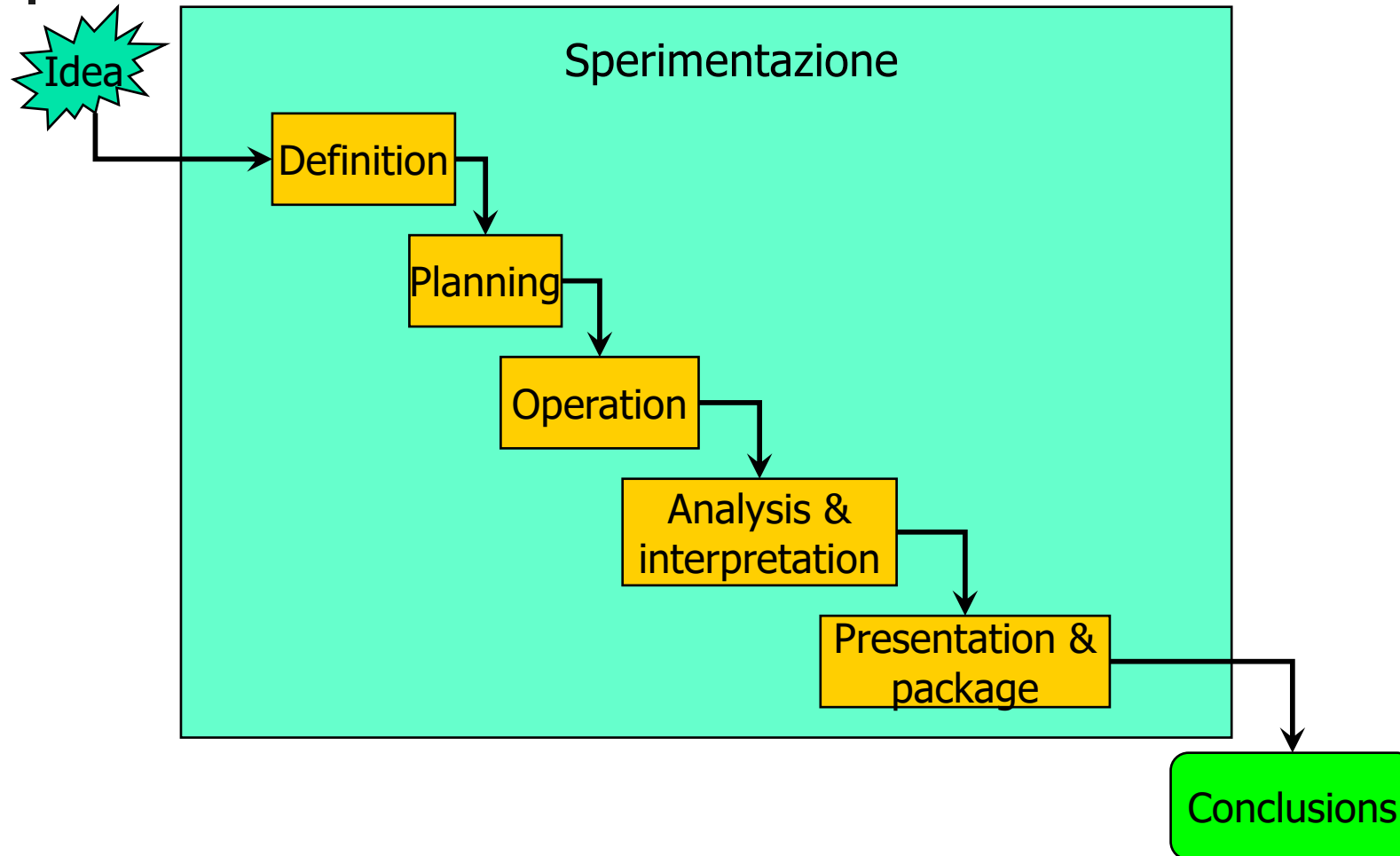


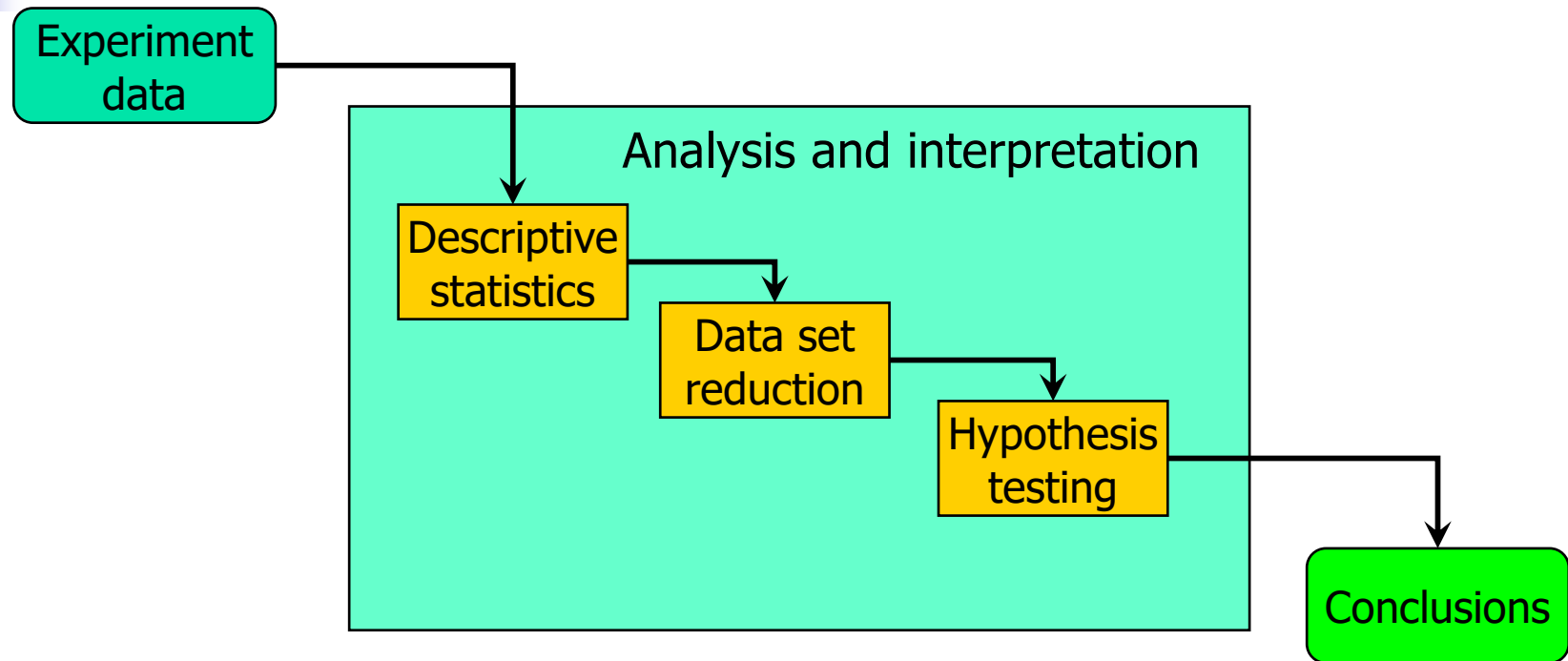
Analysis & Interpretation

Part I

I passi del processo sperimentale

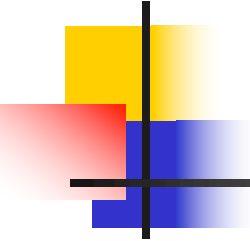


Analysis and interpretation





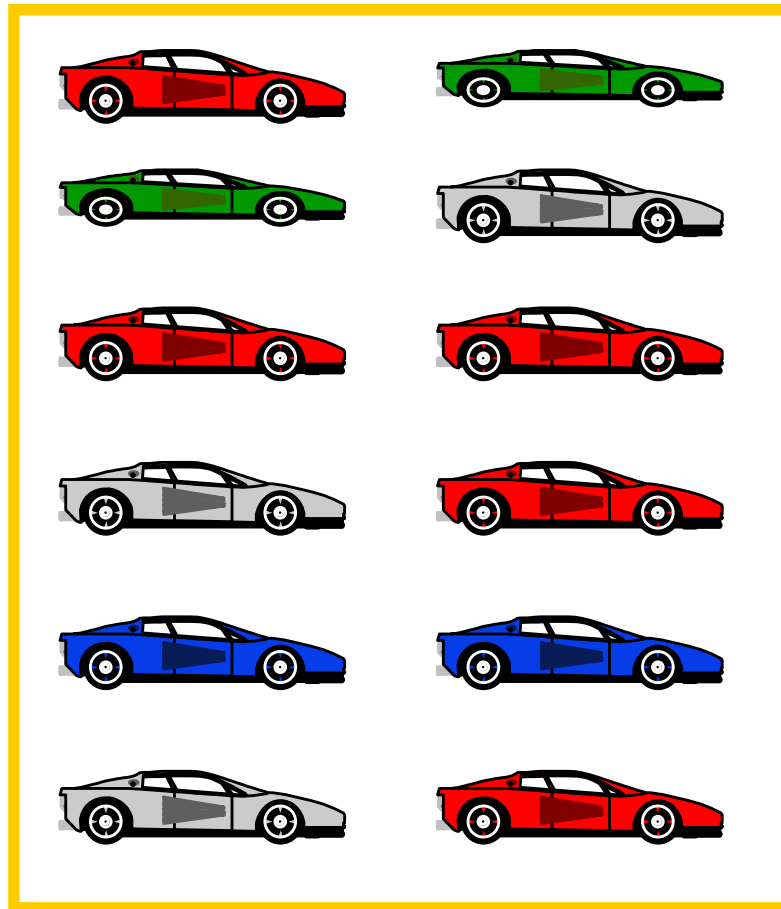
Statistics



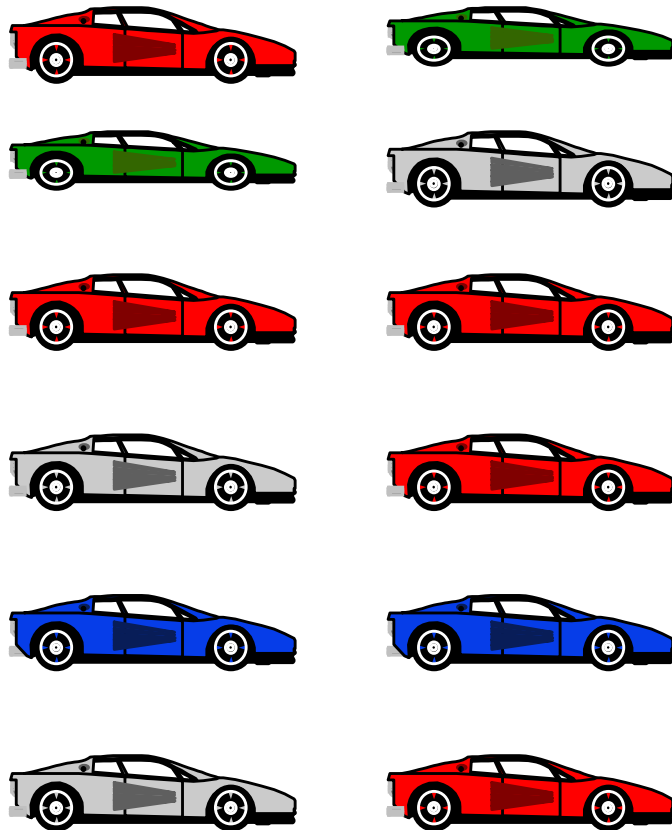
Population Versus Sample

- **Population** — the whole
 - a collection of persons, objects, or items under study
- **Census** — gathering data from the entire population
- **Sample** — a portion of the whole
 - a subset of the population

Population

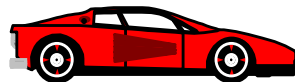
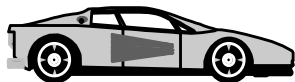
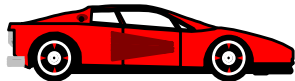


Population and Census Data

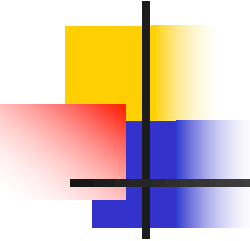


Identifier	Color	MPG
RD1	Red	12
RD2	Red	10
RD3	Red	13
RD4	Red	10
RD5	Red	13
BL1	Blue	27
BL2	Blue	24
GR1	Green	35
GR2	Green	35
GY1	Gray	15
GY2	Gray	18
GY3	Gray	17

Sample and Sample Data



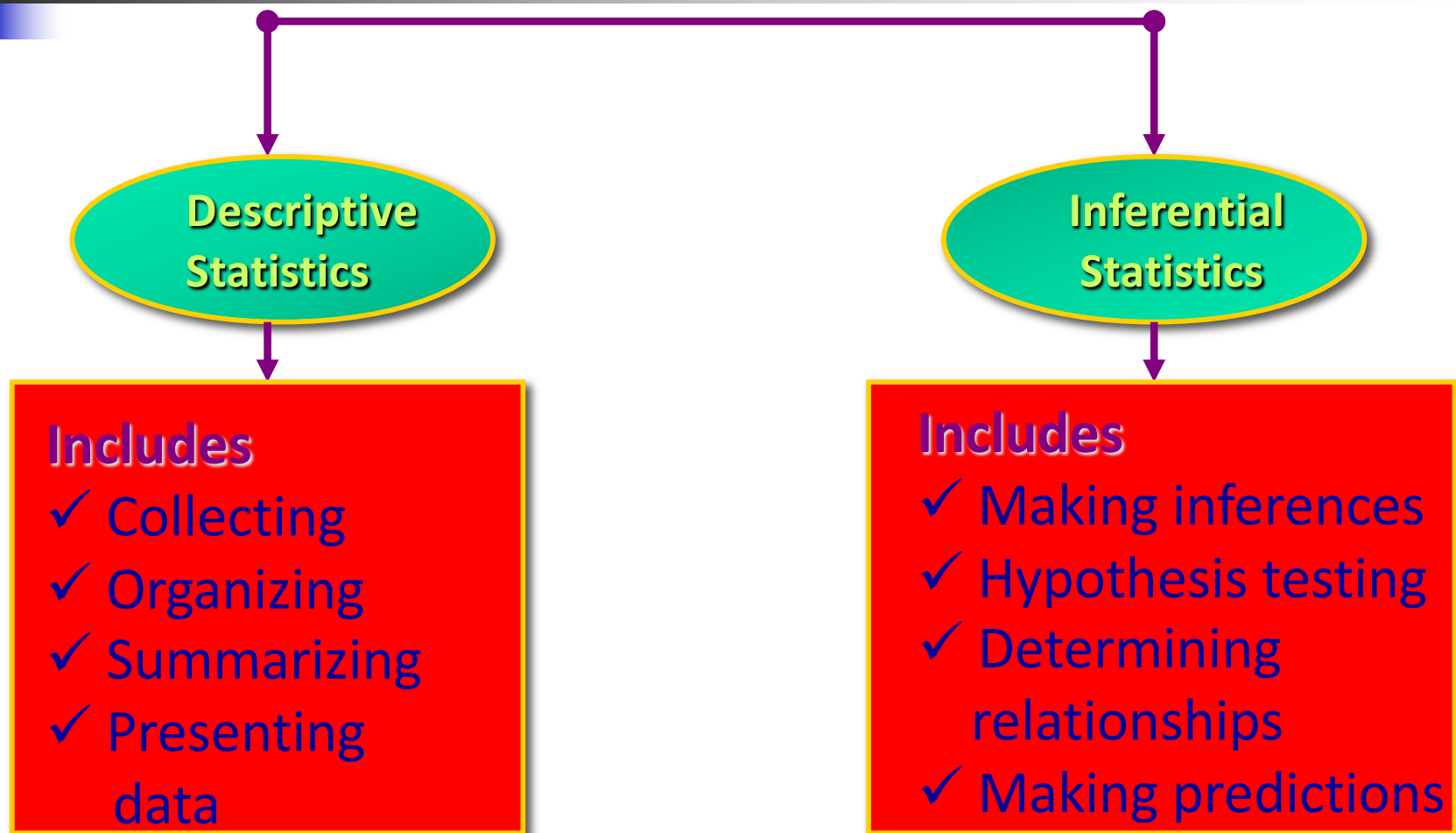
Identifier	Color	MPG
RD2	Red	10
RD5	Red	13
GR1	Green	35
GY2	Gray	18

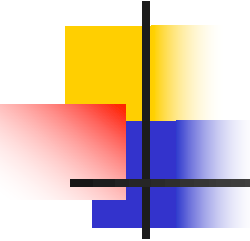


Descriptive vs. Inferential Statistics

- **Descriptive Statistics** — using data gathered on a group to describe or reach conclusions about that same group only
- **Inferential Statistics** — using sample data to reach conclusions about the population from which the sample was taken

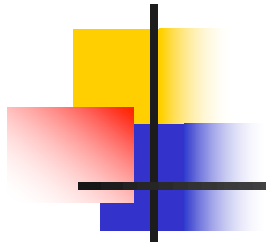
Descriptive vs. Inferential Statistics





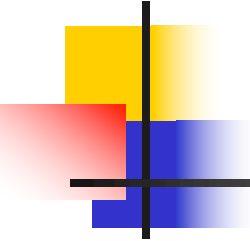
Parameters and statistics

- Both populations and samples have characteristics that are associated with them.
- These are called **parameters** and **statistics** respectively.
- A **parameter** is a characteristic of or a fact about a population.
 - **Example:** The average age for the entire student population on a campus is an example of a parameter.
 - Usually represented by Greek letters



Parameters and statistics

- A **statistic** is a characteristic of or a fact about a sample.
 - Usually represented by Roman letters
 - **Example:** The average ACT math score for a sample of students on a campus is an example of a statistic.

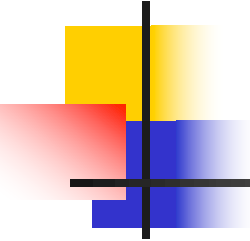


Symbols for Population Parameters

μ denotes population parameter

σ^2 denotes population variance

σ denotes population standard deviation

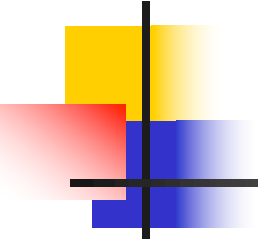


Symbols for Sample Statistics

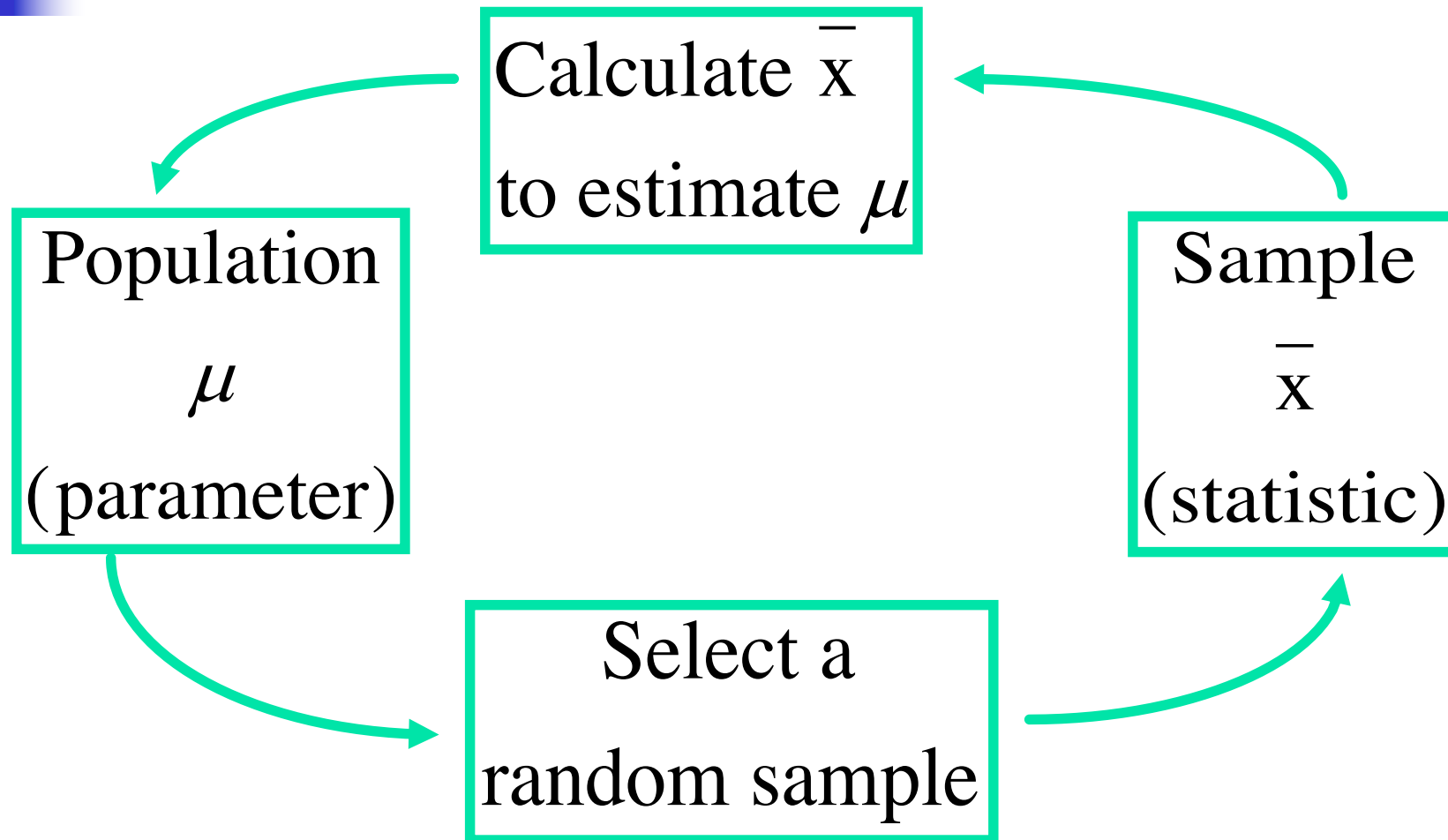
$\bar{x}$ denotes sample mean

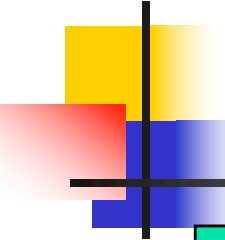
S^2 denotes sample variance

S denotes sample standard deviation



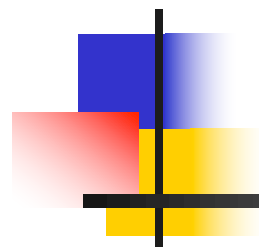
Process of Inferential Statistics



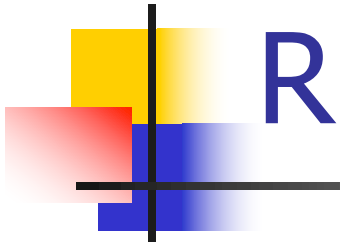


Data Level, Operations, and Statistical Methods

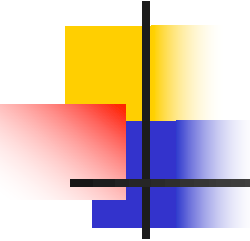
Data Level	Meaningful Operations	Statistical Methods
Nominal	Classifying and Counting	Nonparametric
Ordinal	All of the above plus Ranking	Nonparametric
Interval	All of the above plus Addition, Subtraction, Multiplication, and Division	Parametric
Ratio	All of the above	Parametric



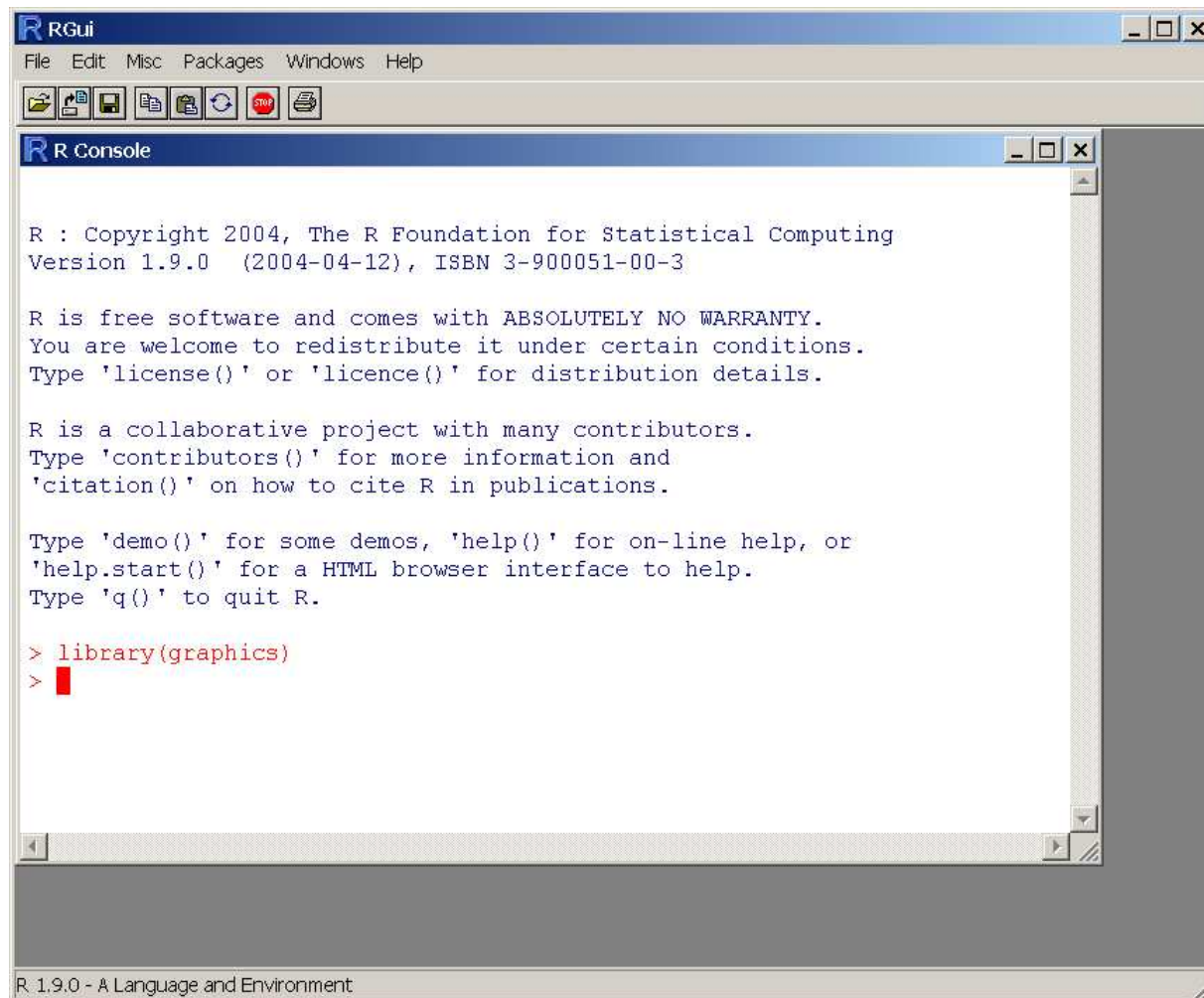
Introduction to R



- R is an integrated suite of software facilities for data manipulation, calculation and graphical display.
 - <http://www.r-project.org>
- Features:
 - Data handling and storage facility
 - A suite of operators for calculations on arrays and matrices
 - A large collection for data analysis
 - Graphical facilities for data analysis
 - A well developed, simple and effective programming language
 - Fully expandible via packages



Getting Started



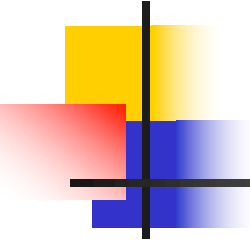
RGui

File Edit Misc Packages Windows Help

R Console

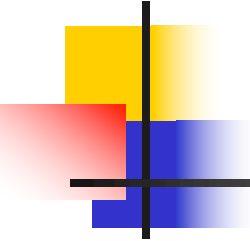
```
R : Copyright 2004, The R Foundation for Statistical Computing  
Version 1.9.0 (2004-04-12), ISBN 3-900051-00-3  
  
R is free software and comes with ABSOLUTELY NO WARRANTY.  
You are welcome to redistribute it under certain conditions.  
Type 'license()' or 'licence()' for distribution details.  
  
R is a collaborative project with many contributors.  
Type 'contributors()' for more information and  
'citation()' on how to cite R in publications.  
  
Type 'demo()' for some demos, 'help()' for on-line help, or  
'help.start()' for a HTML browser interface to help.  
Type 'q()' to quit R.  
  
> library(graphics)  
> 
```

R 1.9.0 - A Language and Environment



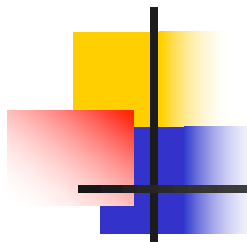
Getting some help

- To get more information on any specific named function, for example `solve`, the command is
 - `> help(solve)`
- An alternative is
 - `> ?solve`
- On most R installations help is available in HTML format by running
 - `> help.start()`
- The examples on a help topic can normally be run by
 - `> example(topic)`



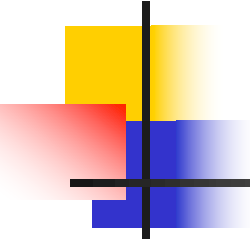
Packages - I

- All R functions and datasets are stored in *packages*.
- To see which packages are installed:
 - `> library()`
- To load a particular package (e.g., the *boot* package)
 - `> library(boot)`
 - Packages are often inter-dependent, and loading one may cause others to be automatically loaded.
 - `available.packages()` function to see the available packages from the net
 - Can also be done from the GUI
- To see which packages are currently loaded
 - `> search()`



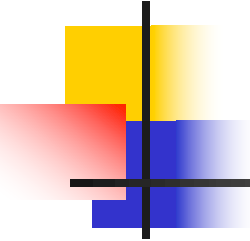
Packages - II

- **Standard (base) packages**
 - part of the R source code.
 - they contain the basic functions, and the standard statistical and graphical functions
- **Contributed packages and CRAN**
 - implement specialized statistical methods,
 - give access to data or hardware,
 - Some (the *recommended* packages) are distributed with every binary distribution of R.
 - Most are available for download from CRAN (<http://cran.r-project.org/>)



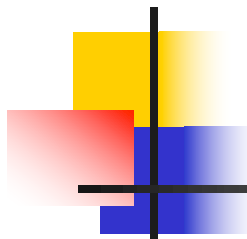
The R language

- **Expression language** with a very simple syntax.
- It is *case sensitive*
- Normally all alphanumeric symbols are allowed, plus . and _
 - a name must start with . or a letter, and if it starts with . the second character must not be a digit.
- **Elementary commands** consist of either *expressions* or *assignments*.
 - If an expression is given as a command, it is evaluated, printed, and the value is lost.
 - An assignment also evaluates an expression and passes the value to a variable but the result is not automatically printed.
- **Commands are separated** either by a semi-colon (;), or by a newline.
- **Elementary commands** can be grouped together into one compound expression by braces ({ and }).
- **Comments** starts a hashmark (#), everything to the end of the line is a comment.
- If a **command is not complete** at the end of a line, R will give a different prompt, by default +



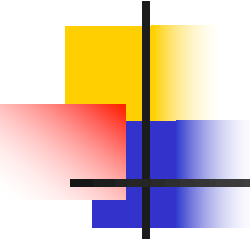
Command files

- If commands are stored on an external file, say `commands.R` in the working directory work, they may be executed at any time in an R session with the command
 - `> source("commands.R")`
- For Windows **Source** is also available on the **File** menu



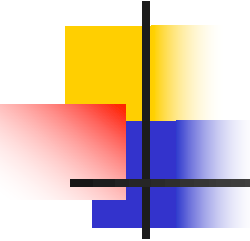
Data permanency and removing objects

- The entities that R creates and manipulates are known as *objects*. These may be variables, arrays of numbers, character strings, functions, or more general structures built from such components.
- During an R session, objects are created and stored by name (we discuss this process in the next session). The R command
- `> objects()` (alternatively, `ls()`) can be used to display the names of the objects which are currently stored within R. The collection of objects currently stored is called the *workspace*.
- To remove objects the function `rm` is available:
- `> rm(x, y, z, ink, junk, temp, foo, bar)`



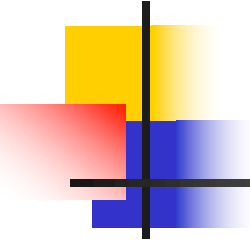
Simple manipulations numbers and vectors

- Vectors and assignment
- Vector arithmetic
- Generating regular sequences
- Logical vectors
- Missing values
- Character vectors
- Index vectors
- Other types of objects



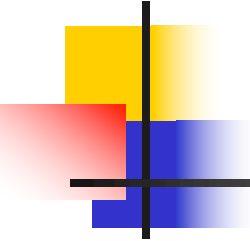
Vectors and assignment

- R operates on named *data structures*.
- **Numeric vector**: ordered collection of numbers.
- To set up a vector named `x`, say, consisting of five numbers, namely 10.4, 5.6, 3.1, 6.4 and 21.7, use the R command
 - `> x <- c(10.4, 5.6, 3.1, 6.4, 21.7)`
 - this is an *assignment* (`<-`) statement using the *function* `c()`
 - `c()` concatenates the arguments into a vector
 - A number occurring by itself in an expression is taken as a vector of length one.
- Notice that the assignment operator (`<-`) is **not** the usual `=` operator



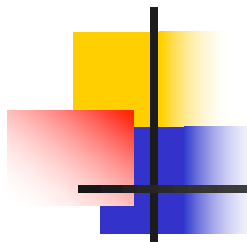
Vectors and assignment

- Assignment can also be made using the function `assign()`
 - `> assign("x", c(10.4, 5.6, 3.1, 6.4, 21.7))`
- Assignments can also be made in the other direction
 - `> c(10.4, 5.6, 3.1, 6.4, 21.7) -> x`
- If an expression is used as a complete command, the value is printed *and lost*.
 - `> 1/x`
 - the reciprocals of the five values would be printed at the terminal (and the value of `x`, of course, unchanged).
- The further assignment
 - `> y <- c(x, 0, x)`
 - creates a vector `y` with 11 entries consisting of two copies of `x` with a zero in the middle place.



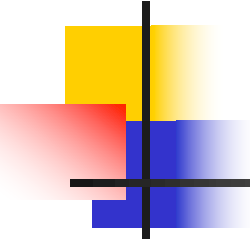
Vectors arithmetics

- Vectors can be used in arithmetic expressions
 - operations are performed element by element.
- Vectors occurring in the same expression need not all be of the same length.
 - the value of the expression is a vector with the same length as the longest vector which occurs in the expression.
 - Shorter vectors in the expression are *recycled*
 - In particular a constant is simply repeated. So with the above assignments the command
 - `> v <- 2*x + y + 1`
 - generates a new vector v of length 11 constructed by adding together, element by element, 2*x repeated 2.2 times, y repeated just once, and 1 repeated 11 times.
- The elementary arithmetic operators are the usual +, -, *, / and ^ for raising to a power



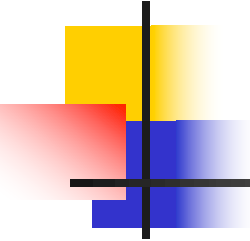
Vectors arithmetics

- In addition all of the common arithmetic functions are available
 - `log`, `exp`, `sin`, `cos`, `tan`, `sqrt`...
 - `max` and `min`
 - `range` is a function whose value is a vector of length two, namely `c(min(x), max(x))`
 - `length(x)` is the number of elements in `x`
 - `sum(x)` gives the total of the elements in `x`
 - `prod(x)` the product of the elements in `x`.



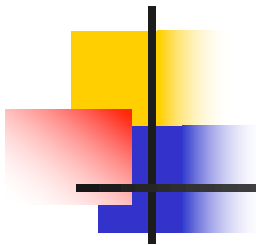
Vectors arithmetics

- Two statistical functions:
 - `mean(x)` which calculates the sample mean
 - `var(x)` which gives $\text{sum}((x - \text{mean}(x))^2) / (\text{length}(x) - 1)$ or sample variance.
- `sort(x)` returns a vector of the same size as `x` with the elements arranged in increasing order;
 - more flexible sorting facilities available (see `order()` or `sort.list()`).
- To work with **complex numbers**, supply an explicit complex part.
 - `sqrt(-17)` will give `NaN` and a warning
 - `sqrt(-17+0i)` will do the computations as complex numbers.



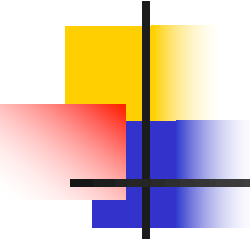
Generating regular sequences

- **1:30** is the vector `c(1, 2, ..., 29, 30)`
 - The construction `30:1` may be used to generate a sequence backwards.
- The function `seq()` is a more general facility
- **Parameters to `seq()`, and to many other R functions, can also be given in named form, in which case the order in which they appear is irrelevant.**
- The first two parameters are: **`from=value`** and **`to=value`**
 - `seq(1,30)`, `seq(from=1, to=30)` and `seq(to=30, from=1)` are all the same as `1:30`.
- The next two parameters are: **`by=value`** and **`length=value`**, which specify a step size and a length for the sequence respectively.
 - `> seq(-5, 5, by=.2) -> s3` generates in `s3` the vector `c(-5.0, -4.8, -4.6, ..., 4.6, 4.8, 5.0)`
- The fifth parameter is: **`along=vector`**, and creates a sequence `1, 2, ..., length(vector)`
 - if used must be the only parameter
 - Empty sequence if the vector is empty



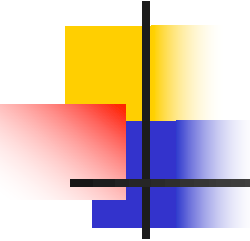
Logical vectors

- The elements of a logical vectors can have the values **TRUE**, **FALSE**, and **NA**.
 - The first two are often abbreviated as T and F, respectively.
 - Note however that T and F are just variables which are set to TRUE and FALSE by default, but are not reserved words
- Logical vectors are generated by *conditions*
 - `> temp <- x > 13` sets temp as a vector of the same length as x with values FALSE corresponding to elements of x where the condition is *not* met and TRUE where it is.
- The relational operators are **<**, **<=**, **>**, **>=**, **==** and **!=**
- In addition if c1 and c2 are logical expressions
 - **c1 & c2** is their intersection (*"and"*),
 - **c1 | c2** is their union (*"or"*), and
 - **!c1** is the negation of c1.
- Logical vectors may be used in ordinary arithmetic
 - FALSE becomes 0 and TRUE becomes 1



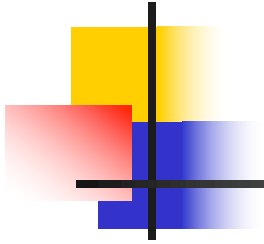
Missing values

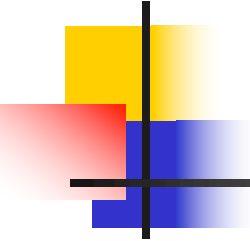
- A place within the vector is reserved using the special value **NA**.
- Any operation on NA produces NA.
- The function **is.na(x)** gives a logical vector of the same size as x with value TRUE if and only if the corresponding element in x is NA.
- Second type of undefined value: **Not a Number, NaN**, values.
 - $> 0/0$
 - $> \text{Inf} - \text{Inf}$
- In summary, **is.na(xx)** is TRUE *both* for NA and NaN values. To differentiate these, **is.nan(xx)** is only TRUE for NaNs.



Character vectors

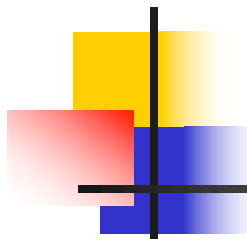
- Used frequently in R, for example as plot labels.
- Entered using either double (") or single (') quotes, printed using double quotes (or without quotes).
- They use C-style escape sequences
- Character vectors may be concatenated into a vector by the `c()` function
- The `paste()` function takes an arbitrary number of arguments and concatenates them one by one into character strings.
 - `> labs <- paste(c("X","Y"), 1:10, sep="")` makes labs into the character vector
`c("X1", "Y2", "X3", "Y4", "X5", "Y6", "X7", "Y8", "X9", "Y10")`
 - Recycling used here!





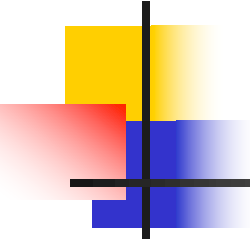
Selecting and modifying subsets of a data set

- Subsets of the elements of a vector may be selected by appending to the name of the vector an *index vector* in square brackets.
- Such index vectors can be:
 - **A logical vector.** Values corresponding to TRUE in the index vector are selected and those corresponding to FALSE omitted
 - `> y <- x[!is.na(x)]` creates an object y which will contain the non-missing values of x,
 - **A vector of positive integral quantities:** the corresponding elements of the vector are selected and concatenated, *in that order*, in the result.
 - `x[6]` is the sixth component of x and
 - `x[1:10]` selects the first 10 elements of x
 - **A vector of negative integral quantities** it specifies the values to be *excluded* rather than included.
 - `y <- x[-(1:5)]` gives y all but the first five elements of x.



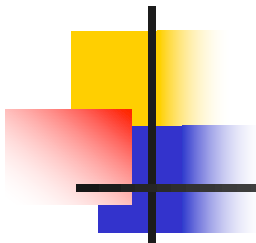
Index vectors; selecting and modifying subsets of a data set

- **A vector of character strings:** only applies where an object has a names attribute to identify its components
 - a sub-vector of the names vector may be used in the same way as the positive integral labels
 - `> fruit <- c(5, 10, 1, 20)`
 - `> names(fruit) <- c("orange", "banana", "apple", "peach")`
 - `> lunch <- fruit[c("apple", "orange")]`



Other types of objects

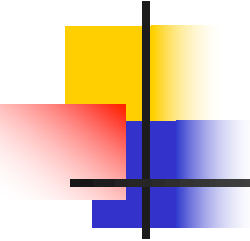
- *matrices* or more generally *arrays* are multi-dimensional generalizations of vectors.
- *factors* provide compact ways to handle categorical data
- *lists* are a general form of vector in which the various elements need not be of the same type
- *data frames* are matrix-like structures, in which the columns can be of different types
- *functions* are themselves objects in R which can be stored in the project's workspace



Factors

- A **factor** is a vector object used to specify a discrete classification (grouping) of the components of other vectors of the same length
- **Example**
 - We have a sample of 30 tax accountants from all the states and territories of Australia and their individual state of origin is specified by a character vector
 - ```
> state <- c("tas", "sa", "qld", "nsw", "nsw", "nt", "wa", "wa", "qld", "vic", "nsw", "vic", "qld", "qld", "sa", "tas", "sa", "nt", "wa", "vic", "qld", "nsw", "nsw", "wa", "sa", "act", "nsw", "vic", "vic", "act")
```
- A **factor** is similarly created using the **factor()** function
  - ```
> statef <- factor(state)
```
- The **print()** function handles factors slightly differently from other objects:
- ```
> statef
```

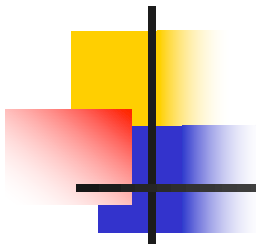
  - ```
[1] tas sa qld nsw nsw nt wa wa qld vic nsw vic qld qld sa
```
 - ```
[16] tas sa nt wa vic qld nsw nsw wa sa act nsw vic vic act
```
  - ```
Levels: act nsw nt qld sa tas vic wa
```
- To find out the levels of a factor the function **levels()** can be used.
 - ```
> levels(statef) [1] "act" "nsw" "nt" "qld" "sa" "tas" "vic" "wa"
```



# Arrays and matrices

---

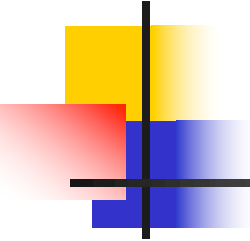
- Arrays
  - Array indexing
  - Index arrays
  - The `array()` function
  - Generalized transpose of an array
- Matrix facilities
- The concatenation function `c()` with arrays
- Frequency tables from factors



# Arrays

---

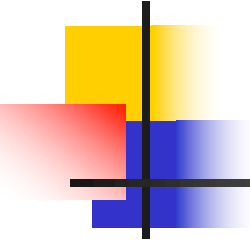
- Multiply subscripted collection of data entries, for example numeric
- A **dimension vector** is a vector of positive integers
  - If its length is  $k$  then the array is  $k$ -dimensional, i.e., a matrix is a 2-dimensional array
  - The values in the dimension vector give the upper limits for each of the  $k$  subscripts
- A **vector can be used by R as an array** only if it has a dimension vector as its *dim* attribute.
  - Suppose, for example,  $z$  is a vector of 1500 elements
  - `> dim(z) <- c(3,5,100)` gives it the *dim* attribute that allows it to be treated as a 3 by 5 by 100 array
- **Data ordering:** “column major order” like in FORTRAN
  - the first subscript moving fastest and the last subscript slowest.
  - If the dimension vector for an array, say  $a$  is `c(3,4,2)` then there are  $3 * 4 * 2 = 24$  entries in  $a$  and the data vector holds them in the order  $a[1,1,1]$ ,  $a[2,1,1]$ , ...,  $a[2,4,2]$ ,  $a[3,4,2]$



# Array indexing

---

- Individual elements of an array may be referenced, as above, by giving the name of the array followed by the subscripts in square brackets, separated by commas
- Subsections of an array may be specified by giving a sequence of *index vectors* in place of subscripts
  - if any index position is given an empty index vector, then the full range of that subscript is taken
  - $a[2,,]$  is a  $4 * 2$  array with dimension vector  $c(4,2)$  and data vector containing the values  $c(a[2,1,1], a[2,2,1], a[2,3,1], a[2,4,1], a[2,1,2], a[2,2,2], a[2,3,2], a[2,4,2])$
  - $a[,,,]$  stands for the entire array



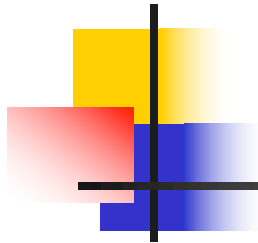
# The array() function

- Arrays can be constructed from vectors by the **array** function
  - `> Z <- array(data_vector, dim_vector)`
  - For example, if the vector `h` contains 24, or fewer, numbers then the command
    - `> Z <- array(h, dim=c(3,4,2))` would use `h` to set up 3 by 4 by 2 array in `Z`.
  - If the size of `h` is exactly 24 the result is the same as
    - `> dim(Z) <- c(3,4,2)`
  - If `h` is shorter than 24, recycling is performed
  - Finally:
    - `> Z <- array(0, c(3,4,2))` makes `Z` an array of all zeros.
- Arrays may be used in arithmetic expressions
  - Result is an array formed by element by element operations
  - The `dim` attributes of operands generally need to be the same, and this becomes the dimension vector of the result



# Generalized transpose of an array

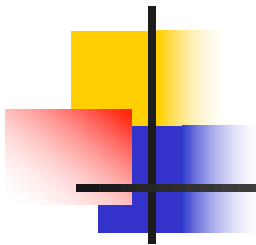
- The function `aperm(a, perm)` may be used to permute an array, `a`
  - The argument `perm` must be a permutation of the integers  $\{1, \dots, k\}$ , where `k` is the number of subscripts in `a`.
  - The result of the function is an array of the same size as `a` but with old dimension given by `perm[j]` becoming the new `j`-th dimension.
  - Example: transposition for matrices.
    - If `A` is a matrix then `B` given by
      - `> B <- aperm(A, c(2,1))` is just the transpose of `A`.
      - For this special case a simpler function `t()` is available, so we could have used `B <- t(A)`.



# Matrix facilities

---

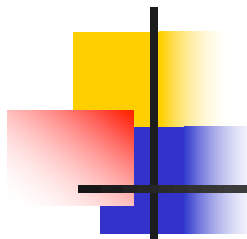
- Multiplication
- Linear equations and inversion
- Singular value decomposition and determinants
- Least squares fitting and the QR decomposition



# Matrix multiplication

---

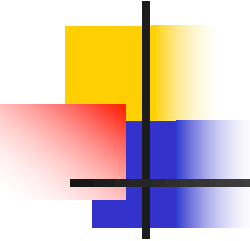
- The operator `%*%` is used for matrix multiplication.
- If, for example, A and B are square matrices of the same size, then
  - `> A * B` is the matrix of element by element products and
  - `> A %*% B` is the matrix product.
- The function `crossprod()` forms "crossproducts", meaning that `crossprod(X, y)` is the same as `t(X) %*% y` but the operation is more efficient.
- The meaning of `diag()` depends on its argument.
  - `diag(v)`, where v is a vector, gives a **diagonal matrix with elements of the vector** as the diagonal entries.
  - On the other hand `diag(M)`, where M is a matrix, gives the **vector of main diagonal entries of M**.
  - Also, somewhat confusingly, if k is a single numeric value then `diag(k)` is the **k by k identity matrix**!



# Linear equations and inversion

---

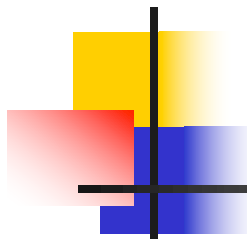
- Solving linear equations is the inverse of matrix multiplication.
- `> b <- A %*% x` only A and b are given, the vector x is the solution of that linear equation system.
- In R,
  - `> solve(A,b)` solves the system, returning x



# From array to vectors

---

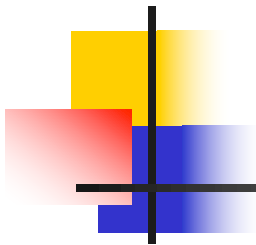
- The way to coerce an array back to a simple vector object is to use `as.vector()`
  - `> vec <- as.vector(X)`
- However a similar result can be achieved by using `c()` with just one argument, simply for this side-effect:
  - `> vec <- c(X)`



# Frequency tables from factors

---

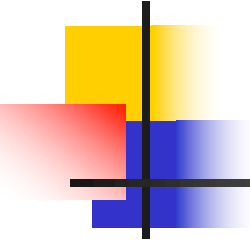
- The function `table()` allows frequency tables to be calculated from equal length factors
  - If there are k category arguments, the result is a k-way array of frequencies
- Suppose, for example, that `statef` is a factor giving the state code for each entry in a data vector. The assignment:
  - `> statefr <- table(statef)` gives in `statefr` a table of frequencies of each state in the sample.
  - The frequencies are ordered and labelled by the levels attribute of the category.



# Lists

---

- An R **list** is an object consisting of an ordered collection of objects known as its **components**.
- There is no particular need for the components to be of the same type
- Here is a simple example of how to make a list:
  - `> Lst <- list(name="Fred", wife="Mary", no.children=3, child.ages=c(4,7,9))`
  - Components are always **numbered** and may always be referred to as such.
  - If Lst is a list, then the function `length(Lst)` gives the number of (top level) components it has.
- Components of lists may also be **named**, and in this case the component may be referred by giving an expression of the form
  - `> name$component_name` for the same thing.
- In the example:
  - `Lst$name` is the same as `Lst[[1]]` and is the string "Fred"
  - `Lst$wife` is the same as `Lst[[2]]` and is the string "Mary"
  - `Lst$child.ages[1]` is the same as `Lst[[4]][1]` and is the number 4

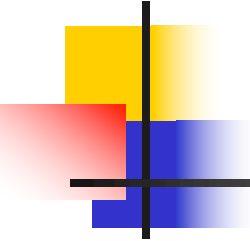


# Reading data from files

---

- The `read.table()` function
- Accessing builtin datasets
- Editing data

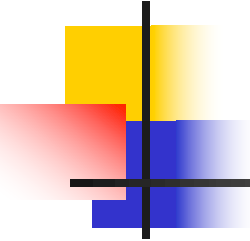




# The read.table() function

- To read an entire data frame directly, the external file will normally have a special form.
- The first line of the file should have a *name* for each variable in the data frame.
- Each additional line of the file has its first item a *row label* and the values for each variable.
- If the file has one fewer item in its first line than in its second, this arrangement is presumed to be in force.
- Example:

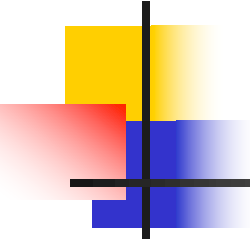
|     | Price | Floor | Area | Rooms | Age | Cent.heat |  |
|-----|-------|-------|------|-------|-----|-----------|--|
| 01  | 52.00 | 111.0 | 830  | 5     | 6.2 | no        |  |
| 02  | 54.75 | 128.0 | 710  | 5     | 7.5 | no        |  |
| 03  | 57.50 | 101.0 | 1000 | 5     | 4.2 | no        |  |
| 04  | 57.50 | 131.0 | 690  | 6     | 8.8 | no        |  |
| 05  | 59.75 | 93.0  | 900  | 5     | 1.9 | yes       |  |
| ... |       |       |      |       |     |           |  |



# The read.table() function

---

- By default numeric items (except row labels) are read as numeric variables and non-numeric variables, such as Cent.heat in the example, as factors.
  - This can be changed if necessary.
  - ```
> HousePrice <- read.table("houses.data")
```
- Often you will want to omit including the row labels directly and use the default labels.
 - ```
> HousePrice <- read.table("houses.data", header=TRUE)
```
  - `header=TRUE` specifies that the first line is a line of headings
  - by implication from the form of the file, no explicit row labels are given



# Read.table syntax

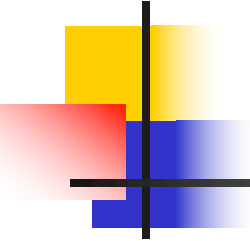
---

```
read.table(file, header = FALSE, sep = "", quote = "\"\"", dec = ".",
...)
```

Esempio:

```
dati <- read.table("/Users/.../AllFold.csv", header = TRUE, sep = ";", dec
= ".")
```

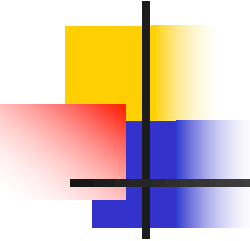
```
read.csv(file, header = TRUE, sep = ",", quote="\"", dec=".",
fill = TRUE, ...)
```



# Accessing builtin datasets

---

- Over fifty datasets are supplied with R,
- Others are available in packages
- To see the list of datasets in the base system use
  - `data()`
- To load one of these use, for example,
  - `data(infert)`
- **Loading data from other R packages**
  - To access data from another package, use the package argument, for example
    - `data(package="stats")`
    - `data(Puromycin, package="stats")`

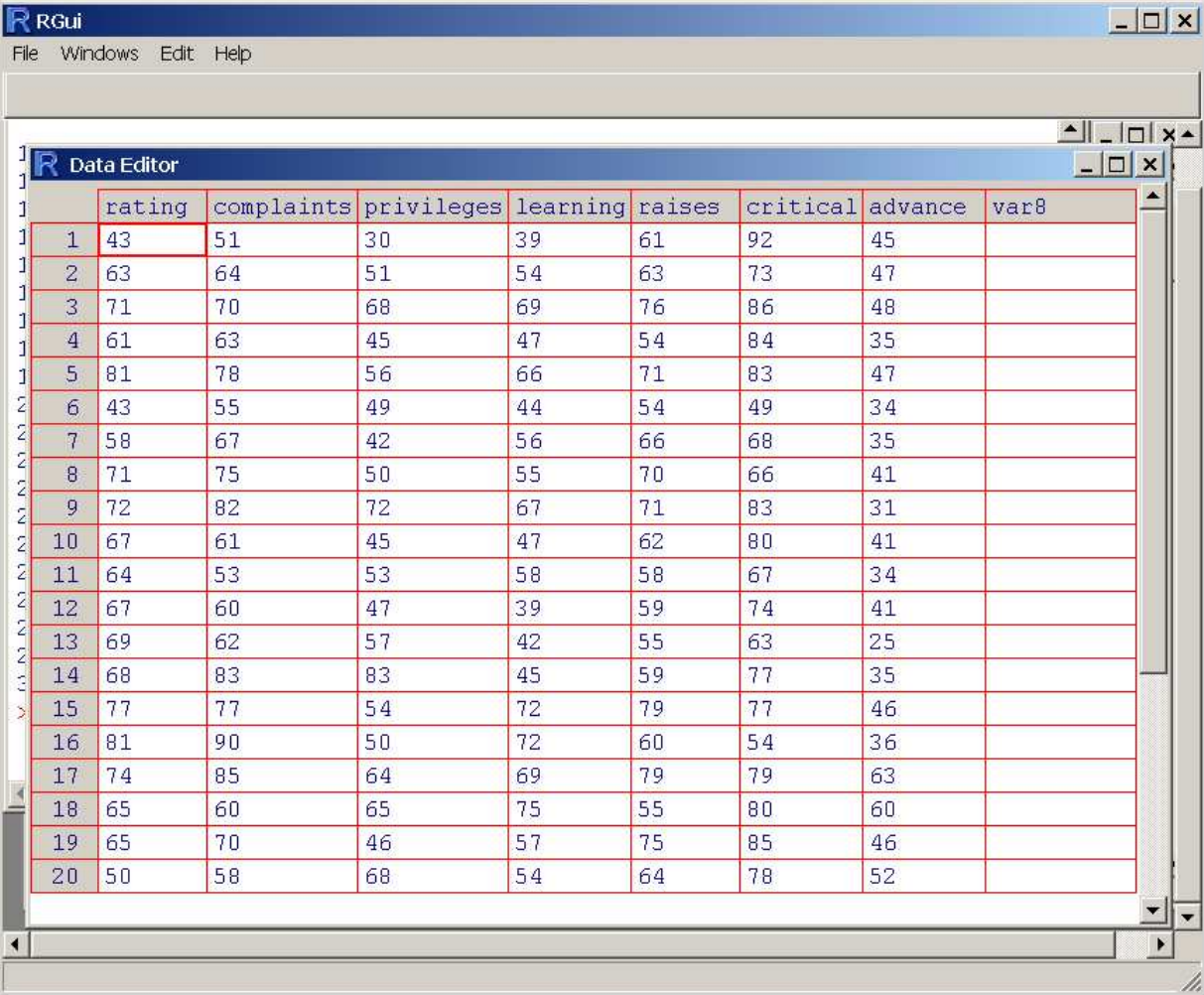


# Editing data - I

---

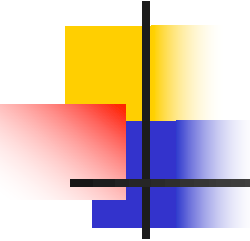
- When invoked on a data frame or matrix, edit brings up a separate spreadsheet-like environment for editing
  - This is useful for making small changes once a data set has been read
  - `> xnew <- edit(xold)` will allow you to edit your data set xold, and on completion the changed object is assigned to xnew.
- Use
  - `> xnew <- edit(data.frame())` to enter new data via the spreadsheet interface.

# Editing data - II



The image shows a screenshot of the RGui Data Editor window. The window has a menu bar with 'File', 'Windows', 'Edit', and 'Help'. The title bar says 'RGui'. The main area is titled 'Data Editor' and displays a table with 20 rows and 9 columns. The columns are labeled 'rating', 'complaints', 'privileges', 'learning', 'raises', 'critical', 'advance', and 'var8'. The rows are numbered 1 through 20 on the left side. The data is as follows:

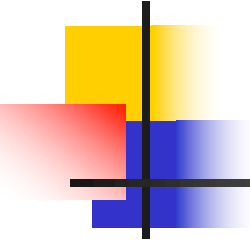
|    | rating | complaints | privileges | learning | raises | critical | advance | var8 |
|----|--------|------------|------------|----------|--------|----------|---------|------|
| 1  | 43     | 51         | 30         | 39       | 61     | 92       | 45      |      |
| 2  | 63     | 64         | 51         | 54       | 63     | 73       | 47      |      |
| 3  | 71     | 70         | 68         | 69       | 76     | 86       | 48      |      |
| 4  | 61     | 63         | 45         | 47       | 54     | 84       | 35      |      |
| 5  | 81     | 78         | 56         | 66       | 71     | 83       | 47      |      |
| 6  | 43     | 55         | 49         | 44       | 54     | 49       | 34      |      |
| 7  | 58     | 67         | 42         | 56       | 66     | 68       | 35      |      |
| 8  | 71     | 75         | 50         | 55       | 70     | 66       | 41      |      |
| 9  | 72     | 82         | 72         | 67       | 71     | 83       | 31      |      |
| 10 | 67     | 61         | 45         | 47       | 62     | 80       | 41      |      |
| 11 | 64     | 53         | 53         | 58       | 58     | 67       | 34      |      |
| 12 | 67     | 60         | 47         | 39       | 59     | 74       | 41      |      |
| 13 | 69     | 62         | 57         | 42       | 55     | 63       | 25      |      |
| 14 | 68     | 83         | 83         | 45       | 59     | 77       | 35      |      |
| 15 | 77     | 77         | 54         | 72       | 79     | 77       | 46      |      |
| 16 | 81     | 90         | 50         | 72       | 60     | 54       | 36      |      |
| 17 | 74     | 85         | 64         | 69       | 79     | 79       | 63      |      |
| 18 | 65     | 60         | 65         | 75       | 55     | 80       | 60      |      |
| 19 | 65     | 70         | 46         | 57       | 75     | 85       | 46      |      |
| 20 | 50     | 58         | 68         | 54       | 64     | 78       | 52      |      |



# Grouping, loops and conditional execution

---

- Grouped expressions
- Control statements
  - Conditional execution
  - Repetitive execution



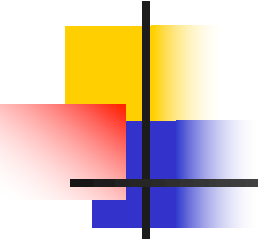
# Grouped expressions

---

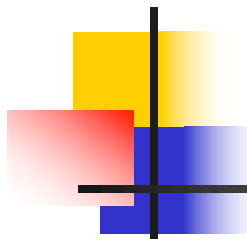
- R is an expression language
  - Any command is an expression which returns a result
- Commands may be grouped together in braces, *{expr\_1; ...; expr\_m}*
  - the value of the group is the result of the last expression in the group evaluated



# Conditional execution: if statements



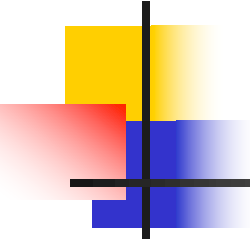
- > *if (expr\_1) expr\_2 else expr\_3*
  - where *expr\_1* must evaluate to a logical value and the result of the entire expression is then evident
- The "short-circuit" operators **&&** and **||** are often used as the condition in an if statement.



# Repetitive execution: for loops, repeat and while

---

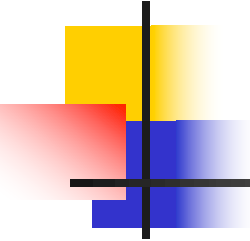
- > *for (name in expr\_1) expr\_2*
  - where *name* is the loop variable
  - *expr\_1* is a vector expression
    - often a sequence like 1:20
  - *expr\_2* is often a grouped expression
  - *expr\_2* is repeatedly evaluated as *name* ranges through the values in the vector result of *expr\_1*



# repeat and while

---

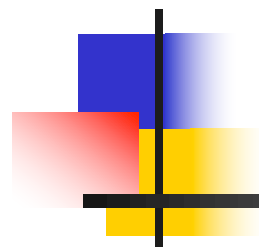
- > **repeat** *expr* statement
- > **while** (*condition*) *expr* statement.
- The **break** statement can be used to terminate any loop
- The **next** statement can be used to discontinue one particular cycle and skip to the "next".



# Writing your own functions

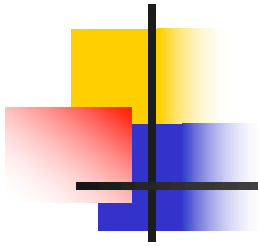
---

- A function is defined by an assignment of the form
  - `> name <- function(arg_1, arg_2, ...) expression`
  - The *expression* is an R expression, (usually a grouped expression)
  - The value of the grouped expression is returned
- A call to the function is in the form  
*name(expr\_1, expr\_2, ...)*
- **Example:**
  - `> multiply<-function(x1,x2){z<-x1*x2;}`
  - `> a<-multiply(2,3)`
  - `> a`
  - `[1] 6`



# Graphs & Charts

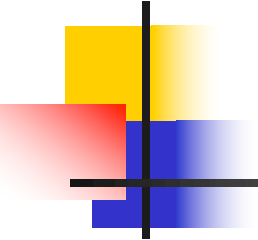
---



# Learning Objectives

---

- Recognize the difference between grouped and ungrouped data
- Construct a frequency distribution
- Construct a histogram, a frequency polygon, an ogive, a pie chart, a stem and leaf plot, a Pareto chart, and a scatter plot

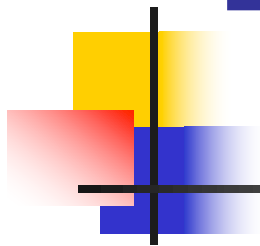


# Ungrouped Versus Grouped Data

---

- Ungrouped data
  - have not been summarized in any way
  - are also called raw data
- Grouped data
  - have been organized into a frequency distribution

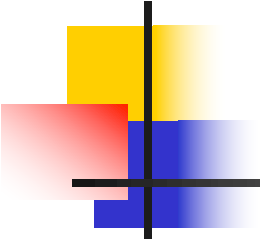
# Example of Ungrouped Data



|    |    |    |    |    |
|----|----|----|----|----|
| 42 | 26 | 32 | 34 | 57 |
| 30 | 58 | 37 | 50 | 30 |
| 53 | 40 | 30 | 47 | 49 |
| 50 | 40 | 32 | 31 | 40 |
| 52 | 28 | 23 | 35 | 25 |
| 30 | 36 | 32 | 26 | 50 |
| 55 | 30 | 58 | 64 | 52 |
| 49 | 33 | 43 | 46 | 32 |
| 61 | 31 | 30 | 40 | 60 |
| 74 | 37 | 29 | 43 | 54 |

**Ages of a Sample of  
Managers from  
Urban Child Care  
Centers in the  
United States**



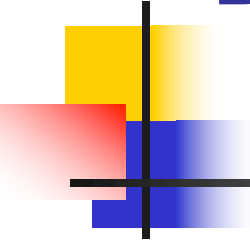


# Frequency Distribution of Child Care Manager's Ages

---

| <u>Class Interval</u> | <u>Frequency</u> |
|-----------------------|------------------|
| 20-under 30           | 6                |
| 30-under 40           | 18               |
| 40-under 50           | 11               |
| 50-under 60           | 11               |
| 60-under 70           | 3                |
| 70-under 80           | 1                |

# Data Range

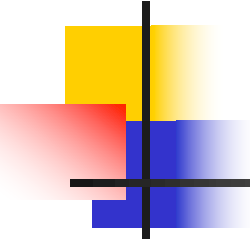


|    |    |    |    |    |
|----|----|----|----|----|
| 42 | 26 | 32 | 34 | 57 |
| 30 | 58 | 37 | 50 | 30 |
| 53 | 40 | 30 | 47 | 49 |
| 50 | 40 | 32 | 31 | 40 |
| 52 | 28 | 23 | 35 | 25 |
| 30 | 36 | 32 | 26 | 50 |
| 55 | 30 | 58 | 64 | 52 |
| 49 | 33 | 43 | 46 | 32 |
| 61 | 31 | 30 | 40 | 60 |
| 74 | 37 | 29 | 43 | 54 |

$$\begin{aligned}\text{Range} &= \text{Largest} - \text{Smallest} \\ &= 74 - 23 \\ &= 51\end{aligned}$$

**Smallest**

**Largest**

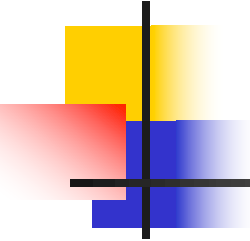


# Number of Classes and Class Width

- **The number of classes** should be between 5 and 15.
  - Fewer than 5 classes cause excessive summarization.
  - More than 15 classes leave too much detail.
- **Class Width**
  - Divide the range by the number of classes for an approximate class width
  - Round up to a convenient number

$$\text{Approximate Class Width} = \frac{51}{6} = 8.5$$

$$\text{Class Width} = 10$$



# Sturge's rule

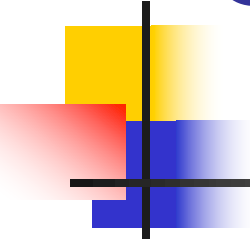
---

- According to Sturge, the number of classes is given by:

$$k = 1 + \log_2 n$$

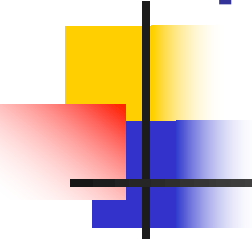
- **Be careful!** If data are not normally distributed → more classes are needed
- **R:** see command
- **`nclass.Sturges(x)`**

# Class Midpoint


$$\begin{aligned}\text{Class Midpoint} &= \frac{\text{beginning class endpoint} + \text{ending class endpoint}}{2} \\ &= \frac{30 + 40}{2} \\ &= 35\end{aligned}$$

$$\begin{aligned}\text{Class Midpoint} &= \text{class beginning point} + \frac{1}{2} \text{class width} \\ &= 30 + \frac{1}{2}(10) \\ &= 35\end{aligned}$$

# Relative Frequency



| <u>Class Interval</u> | <u>Frequency</u> | <u>Relative Frequency</u> |
|-----------------------|------------------|---------------------------|
| 20-under 30           | 6                | .12                       |
| 30-under 40           | 18               | .36                       |
| 40-under 50           | 11               | .22                       |
| 50-under 60           | 11               | .22                       |
| 60-under 70           | 3                | .06                       |
| 70-under 80           | 1                | .02                       |
| Total                 | 50               | 1.00                      |

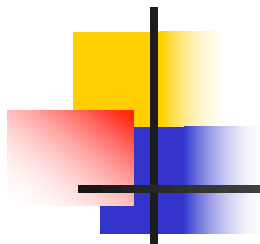
Diagram illustrating the calculation of Relative Frequency:

- For the 20-under 30 class:  $\frac{6}{50} = .12$
- For the 30-under 40 class:  $\frac{18}{50} = .36$
- For the 40-under 50 class:  $\frac{11}{50} = .22$
- For the 50-under 60 class:  $\frac{11}{50} = .22$
- For the 60-under 70 class:  $\frac{3}{50} = .06$
- For the 70-under 80 class:  $\frac{1}{50} = .02$
- Total:  $\frac{50}{50} = 1.00$

# Cumulative Frequency

| Cumulative<br>Class Interval | Frequency | Frequency    |
|------------------------------|-----------|--------------|
| 20-under 30                  | 6         | 6            |
| 30-under 40                  | 18        | 18 + 6 = 24  |
| 40-under 50                  | 11        | 11 + 24 = 35 |
| 50-under 60                  | 11        | 46           |
| 60-under 70                  | 3         | 49           |
| 70-under 80                  | 1         | 50           |
| Total                        | 50        |              |

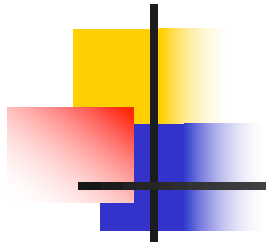
# Class Midpoints, Relative Frequencies, and Cumulative Frequencies



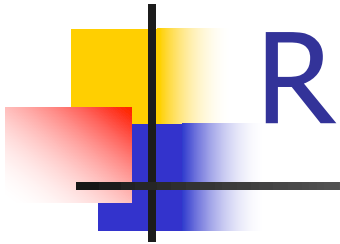
| <u>Class Interval</u> | <u>Frequency</u> | <u>Midpoint</u> | <u>Relative Frequency</u> | <u>Cumulative Frequency</u> |
|-----------------------|------------------|-----------------|---------------------------|-----------------------------|
| 20-under 30           | 6                | 25              | .12                       | 6                           |
| 30-under 40           | 18               | 35              | .36                       | 24                          |
| 40-under 50           | 11               | 45              | .22                       | 35                          |
| 50-under 60           | 11               | 55              | .22                       | 46                          |
| 60-under 70           | 3                | 65              | .06                       | 49                          |
| 70-under 80           | 1                | 75              | .02                       | 50                          |
| Total                 | 50               |                 | 1.00                      |                             |



# Cumulative Relative Frequencies



| <u>Class Interval</u> | <u>Frequency</u> | <u>Relative Frequency</u> | <u>Cumulative Frequency</u> | <u>Cumulative Relative Frequency</u> |
|-----------------------|------------------|---------------------------|-----------------------------|--------------------------------------|
| 20-under 30           | 6                | .12                       | 6                           | .12                                  |
| 30-under 40           | 18               | .36                       | 24                          | .48                                  |
| 40-under 50           | 11               | .22                       | 35                          | .70                                  |
| 50-under 60           | 11               | .22                       | 46                          | .92                                  |
| 60-under 70           | 3                | .06                       | 49                          | .98                                  |
| 70-under 80           | 1                | .02                       | 50                          | 1.00                                 |
| Total                 | 50               | 1.00                      |                             |                                      |



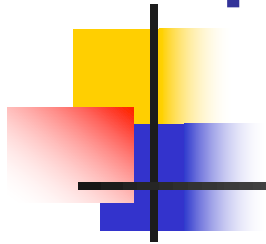
- Given
  - `x<-c(1,3,5,1,2,3,4,6,2,4,1,6,3,2)`
- Absolute frequencies:
  - `f.abs<-table(x)`
- Relative frequencies:
  - `f.rel<-table(x)/length(x)`
  - `f.relperc<-f.rel*100`
- Cumulative frequencies:
  - `f.cum<-cumsum(table(x))`
  - `f.rel.cum<-f.cum/length(x)`
  - `f.rel.cum.perc<-f.rel.cum*100`

# Common Statistical Graphs



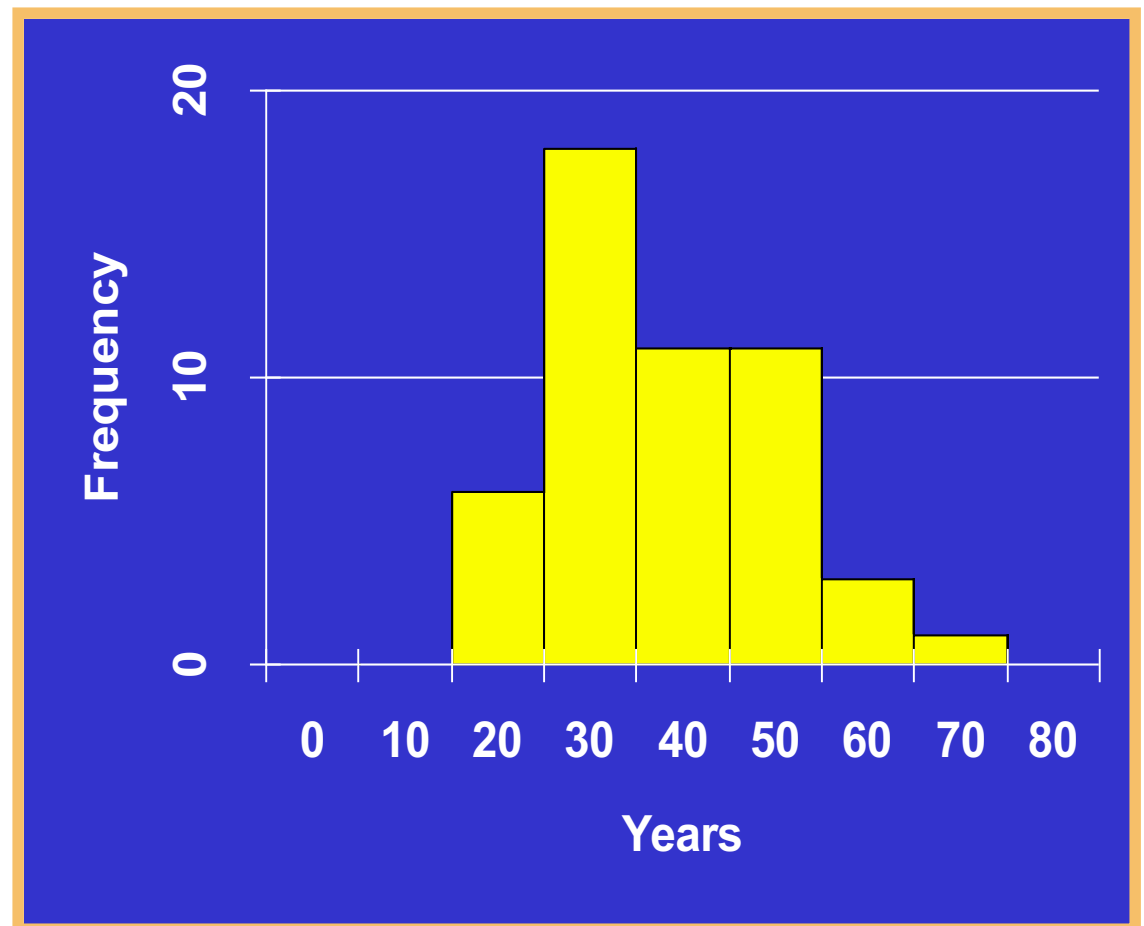
- **Histogram** -- vertical bar chart of frequencies
- **Frequency Polygon** -- line graph of frequencies
- **Ogive** -- line graph of cumulative frequencies
- **Pie Chart** -- proportional representation for categories of a whole
- **Stem and Leaf Plot**
- **Pareto Chart**
- **Scatter Plot**

# Histogram



## Class Interval   Frequency

|             |    |
|-------------|----|
| 20-under 30 | 6  |
| 30-under 40 | 18 |
| 40-under 50 | 11 |
| 50-under 60 | 11 |
| 60-under 70 | 3  |
| 70-under 80 | 1  |



# Histogram Construction

## Class Interval   Frequency

20-under 30

6

30-under 40

18

40-under 50

11

50-under 60

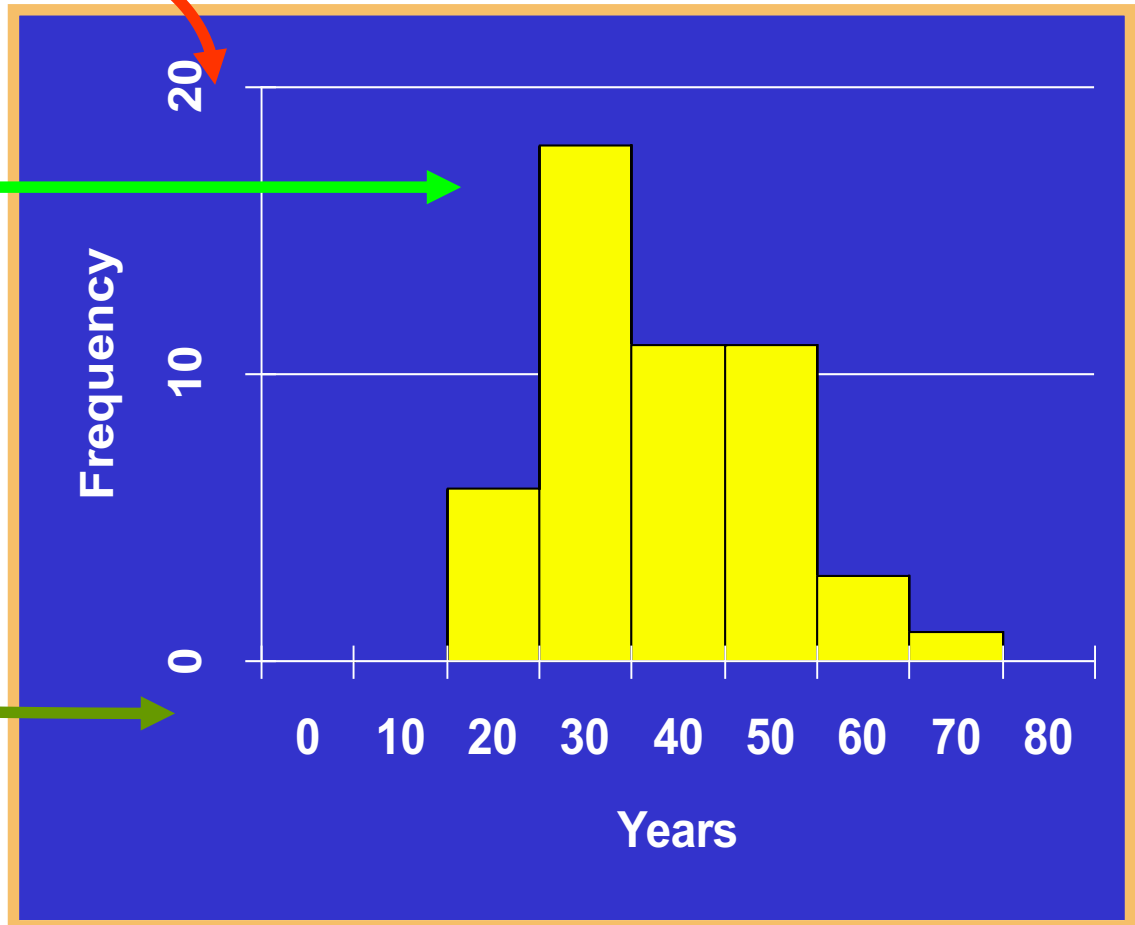
11

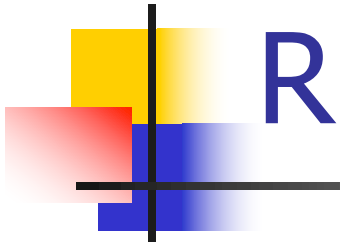
60-under 70

3

70-under 80

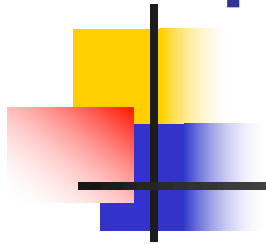
1





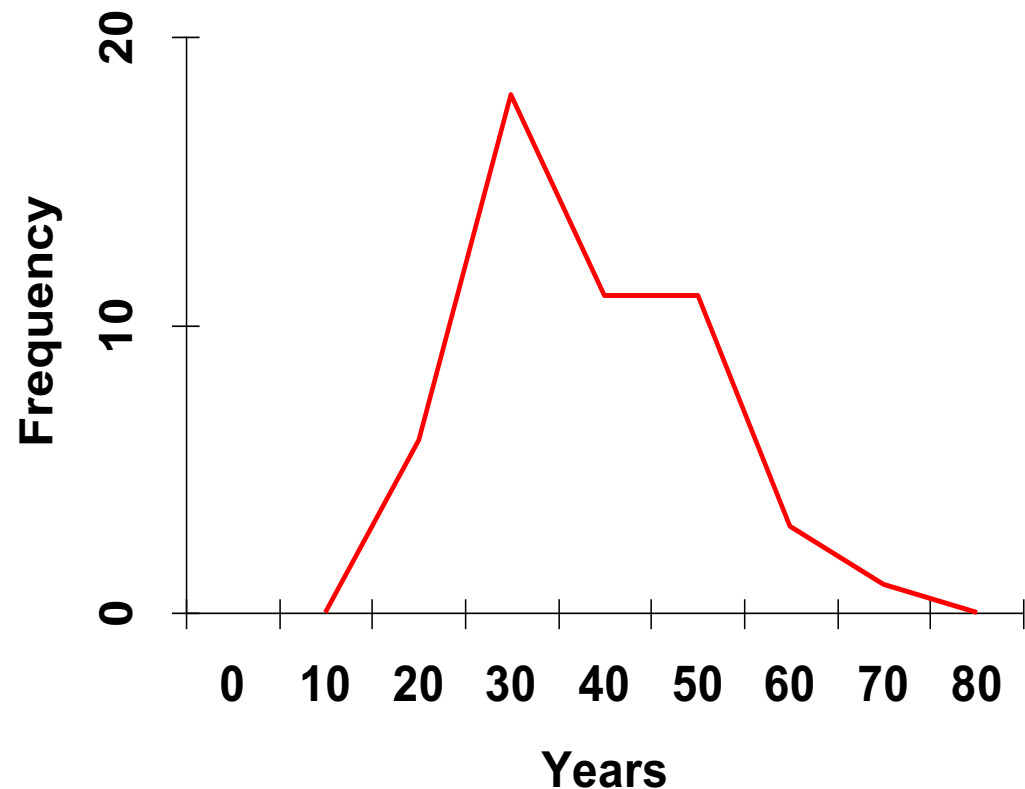
- `hist(x, breaks = "Sturges", ...)`
- `-----> help(hist)`
  
- Components:
  - `breaks` the  $n+1$  cell boundaries
  - `counts`  $n$  integers; for each cell, the number of `x[]` inside.
  - `density` values  $f^{\wedge}(x[i])$ , as estimated density values. If `all(diff(breaks) == 1)`, they are the relative frequencies `counts/n` and in general satisfy  $\sum[i; f^{\wedge}(x[i]) (b[i+1]-b[i])] = 1$ , where  $b[i] = \text{breaks}[i]$ .
  - `mids` the  $n$  cell midpoints.
  - `xname` a character string with the actual `x` argument name.
  - `equidist` logical, indicating if the distances between breaks are all the same.

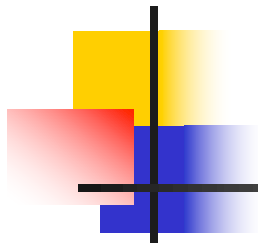
# Frequency Polygon



## Class Interval   Frequency

|             |    |
|-------------|----|
| 20-under 30 | 6  |
| 30-under 40 | 18 |
| 40-under 50 | 11 |
| 50-under 60 | 11 |
| 60-under 70 | 3  |
| 70-under 80 | 1  |





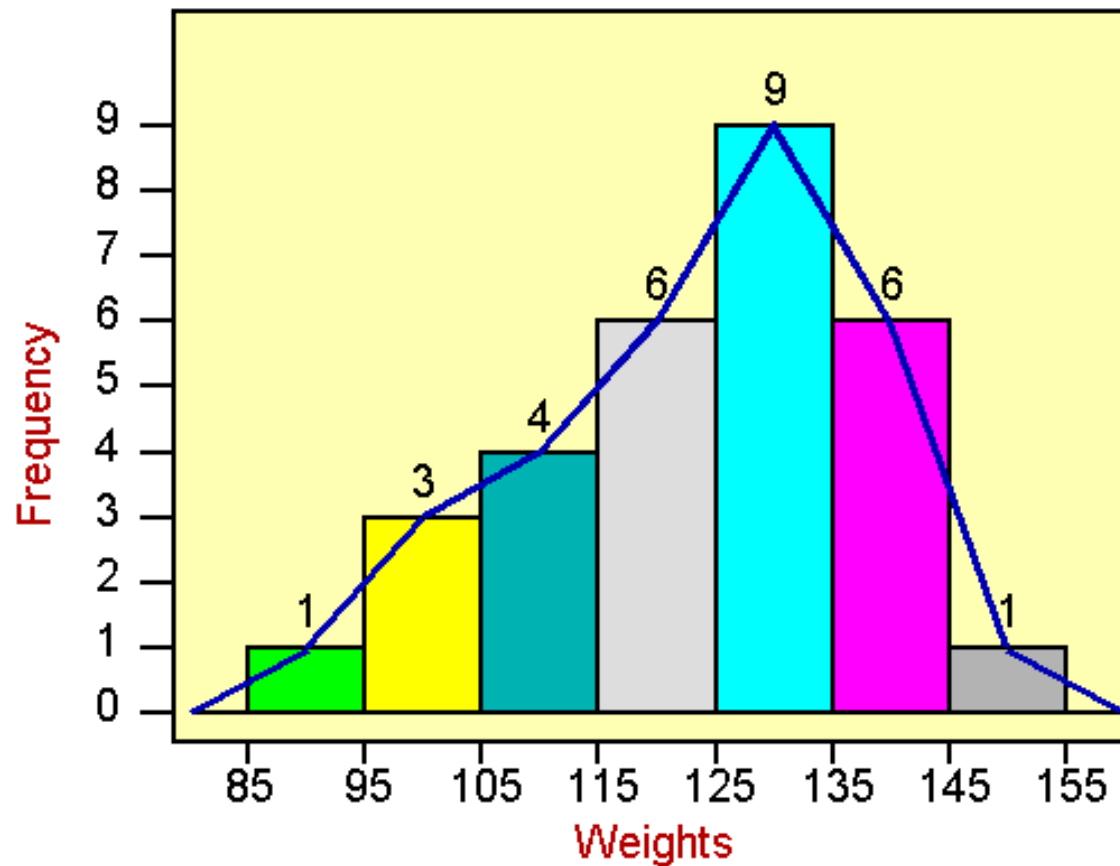
# Frequency Polygon

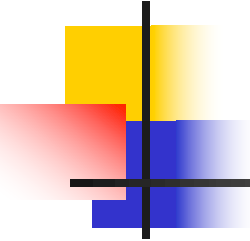
---

- The frequency polygon is superimposed on the histogram.
- The polygon is mound-shaped.
- This indicates that the shape of the population from which the sample was taken is mound shaped.
- The line segments pass through the mid points at the top of the rectangles.
- The polygon is tied down at both ends.



# Frequency Polygon (Cont.)





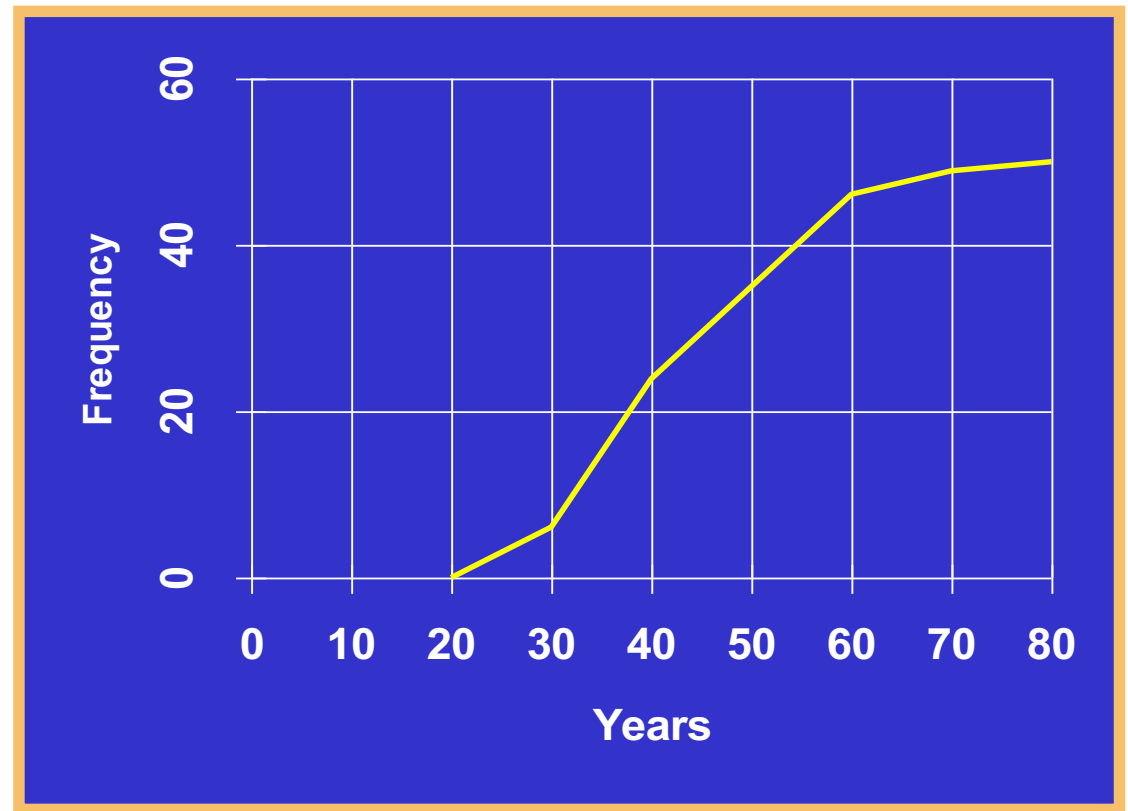
# Frequency Polygon: R

---

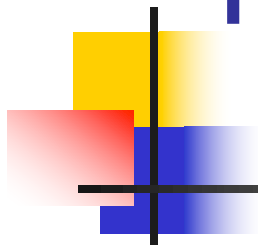
- `c<-hist(x)`
- `plot(c$counts,type="l")`
- `-----> help(plot)`

# Ogive

| <u>Class Interval</u> | <u>Cumulative Frequency</u> |
|-----------------------|-----------------------------|
| 20-under 30           | 6                           |
| 30-under 40           | 24                          |
| 40-under 50           | 35                          |
| 50-under 60           | 46                          |
| 60-under 70           | 49                          |
| 70-under 80           | 50                          |



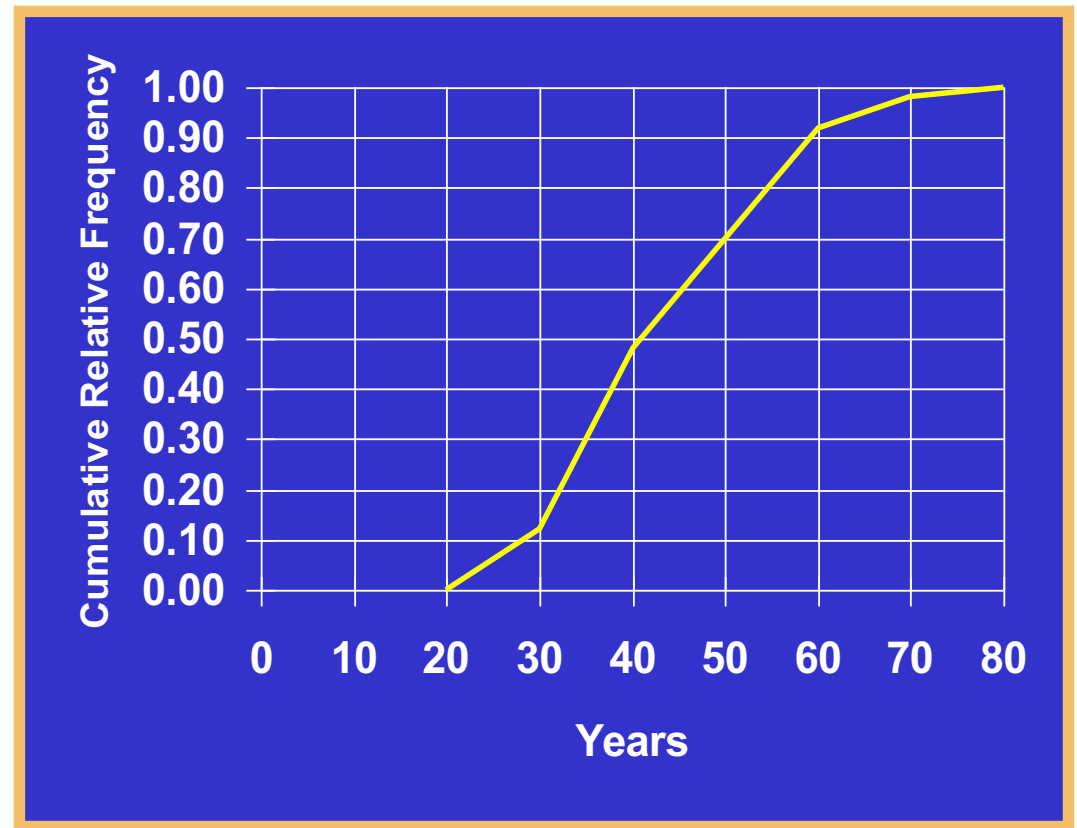
# Relative Frequency Ogive

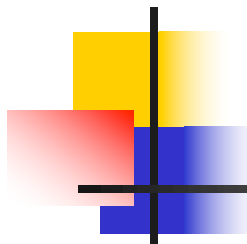


## Cumulative Relative Frequency

### Class Interval

|             |      |
|-------------|------|
| 20-under 30 | .12  |
| 30-under 40 | .48  |
| 40-under 50 | .70  |
| 50-under 60 | .92  |
| 60-under 70 | .98  |
| 70-under 80 | 1.00 |





# Ogive: R

---

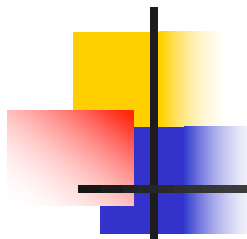
- Absolute:

- `> h <- hist(x, plot = FALSE)`
- `> cu <- cumsum(h$count)`
- `> plot(cu, type = "l")`

- Relative:

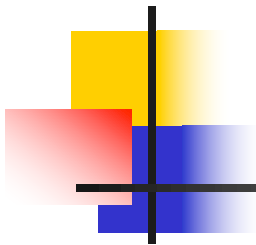
- `> cu.r <- cumsum(h$count)/length(x)`

# Complaints by Amtrak Passengers



| COMPLAINT          | NUMBER | PROPORTION | DEGREES |
|--------------------|--------|------------|---------|
| Stations, etc.     | 28,000 | .40        | 144.0   |
| Train              | 14,700 | .21        | 75.6    |
| Performance        |        |            |         |
| Equipment          | 10,500 | .15        | 50.4    |
| Personnel          | 9,800  | .14        | 50.6    |
| Schedules,<br>etc. | 7,000  | .10        | 36.0    |
| Total              | 70,000 | 1.00       | 360.0   |

- `>y<-c(28000,14700,10500,9800,7000)`
- `> x<-c("stations", "train","perf.equip","personnel", "schedule etc.")`
- `>pie(y,labels=x)`

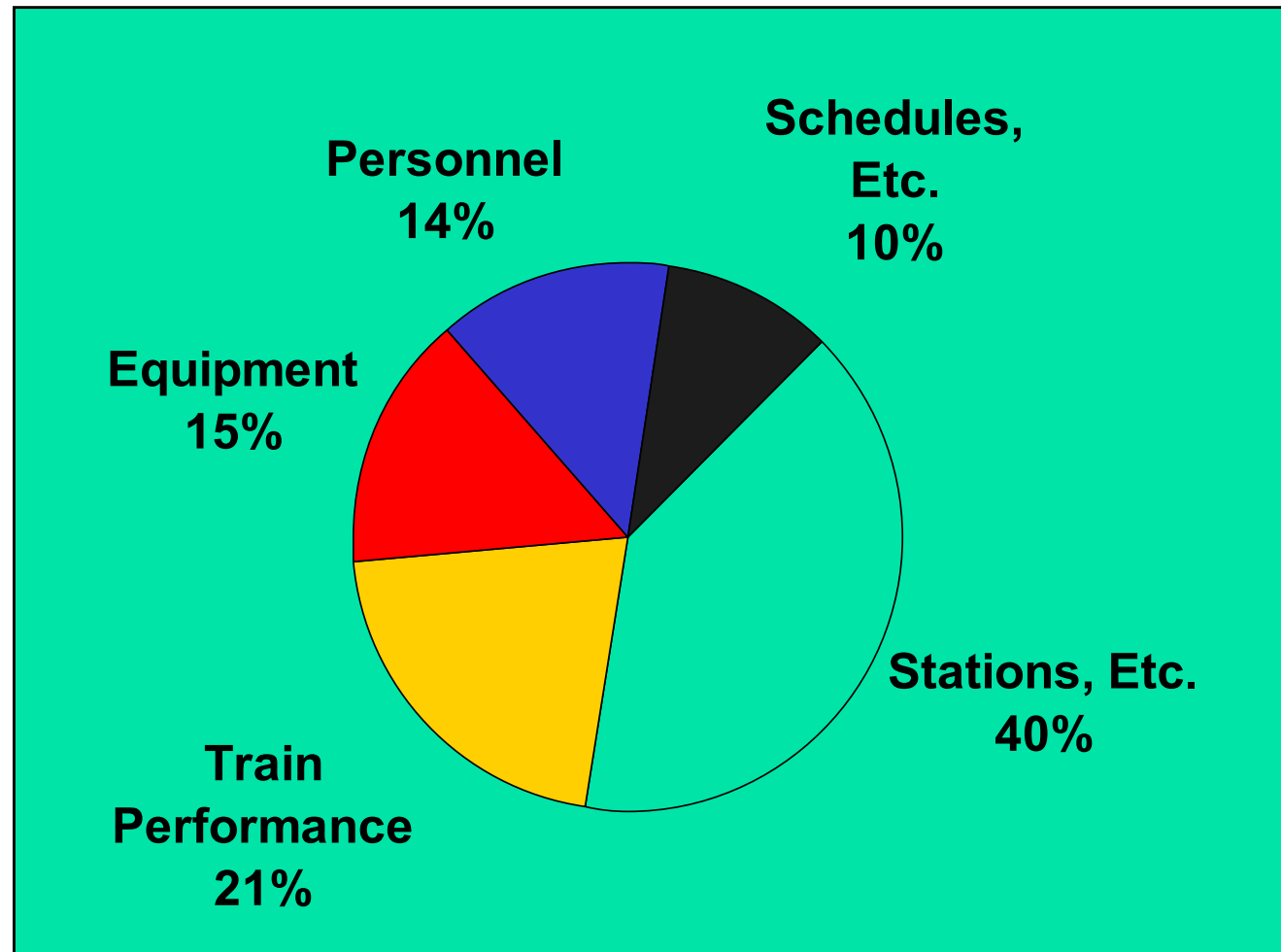


# Pie charts

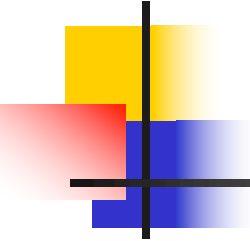
---

- **What is a pie graph (chart)?** A pie graph is a circular display that is divided into sectors (**classes**) according to the percentage of data values in each class.
- **Note:** A pie chart allows us to observe the proportions of the classes relative to the entire data set.
- Pie charts are readily used to display qualitative data.

# Complaints by Amtrak Passengers







# Steam and leaf plot

---

- **What is a stem-and-leaf plot?** A stem-and-leaf plot is a data plot that uses part of a data value as the **stem** to form groups or classes and part of the data value as the **leaf**.
- **Note:** A stem-and-leaf plot has an advantage over a grouped frequency distribution, since a stem-and-leaf plot retains the actual data by showing them in graphic form.

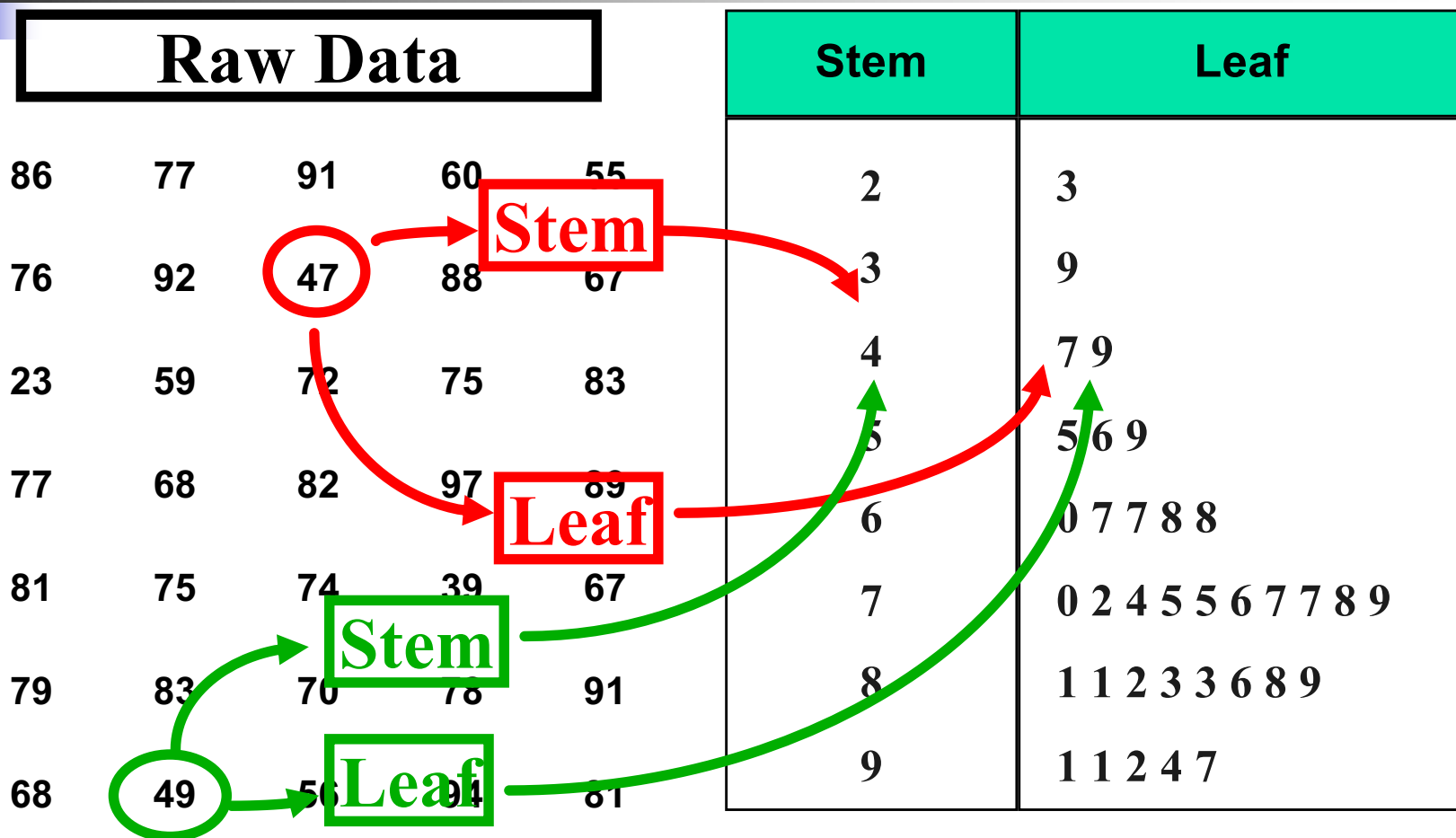
# Safety Examination Scores for Plant Trainees

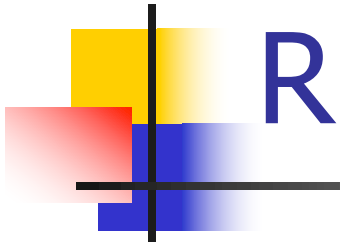
## Raw Data

|    |    |    |    |    |
|----|----|----|----|----|
| 86 | 77 | 91 | 60 | 55 |
| 76 | 92 | 47 | 88 | 67 |
| 23 | 59 | 72 | 75 | 83 |
| 77 | 68 | 82 | 97 | 89 |
| 81 | 75 | 74 | 39 | 67 |
| 79 | 83 | 70 | 78 | 91 |
| 68 | 49 | 56 | 94 | 81 |

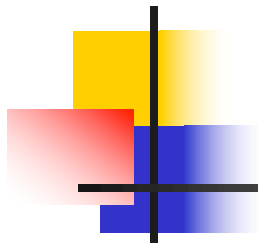
| Stem | Leaf                |
|------|---------------------|
| 2    | 3                   |
| 3    | 9                   |
| 4    | 7 9                 |
| 5    | 5 6 9               |
| 6    | 0 7 7 8 8           |
| 7    | 0 2 4 5 5 6 7 7 8 9 |
| 8    | 1 1 2 3 3 6 8 9     |
| 9    | 1 1 2 4 7           |

# Construction of Stem and Leaf Plot





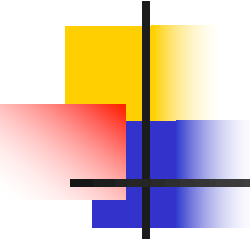
- `stem(x, scale = 1, width = 80, atom = 1e-08)`
  - stem produces a stem-and-leaf plot of the values in x
- Example
  - `> x<-c(2,4,6,8,12,16,24,35,12,35,32,11,23)`
  - `> stem(x,scale=1)`
  - The decimal point is 1 digit(s) to the right of the |
  - 0 | 2468
  - 1 | 1226
  - 2 | 34
  - 3 | 255



# Time series graph

---

- **What is a time series graph?** A time series graph is a plot which displays data that are observed over a given period of time.
- **Note:** From a time series graph, one can observe and analyze the behavior of the data over time.



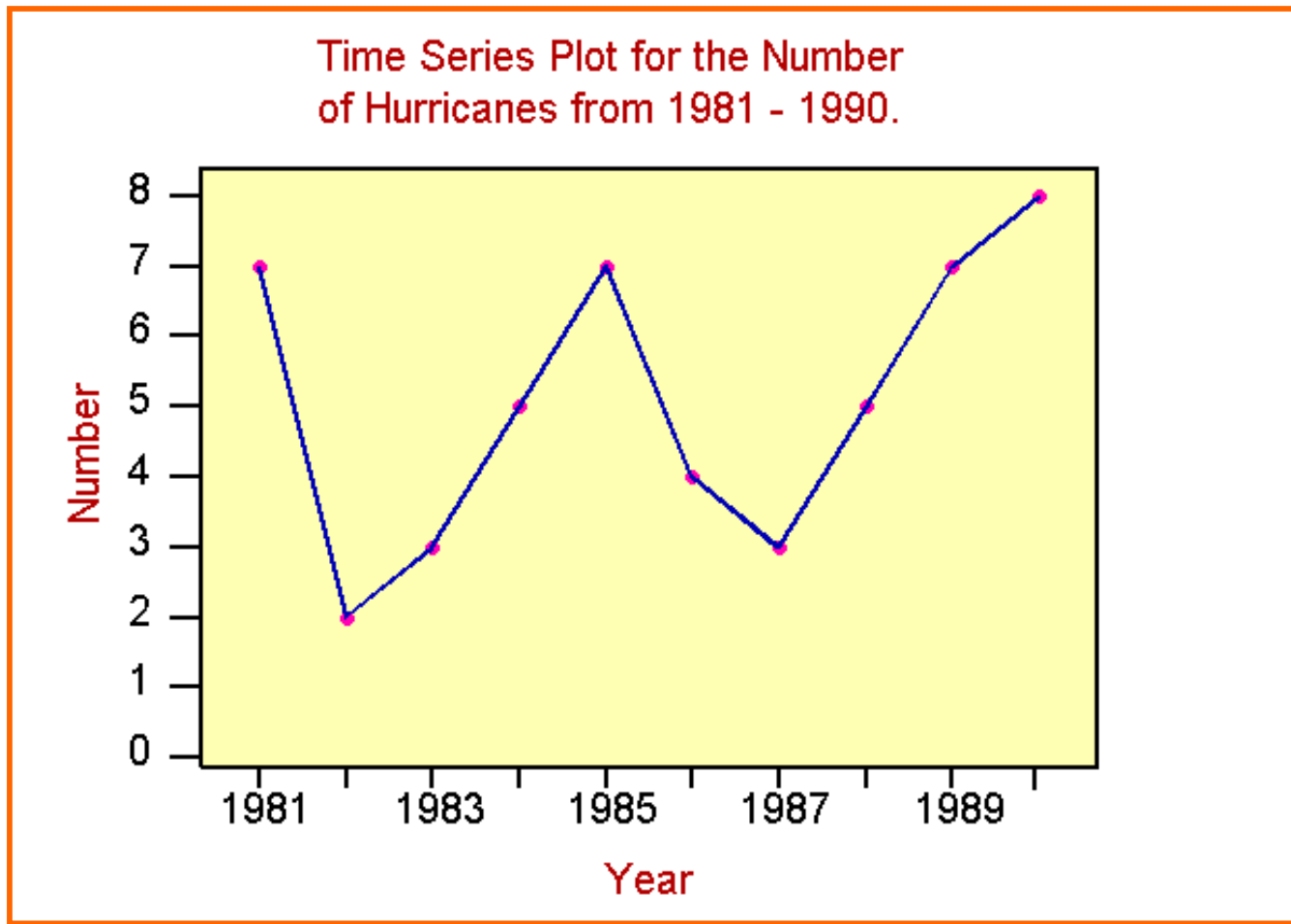
# Time series graph - Example

- **Example:** The following table gives the number of hurricanes for the years 1981 to 1990.

| Year   | 1981 | 1982 | 1983 | 1984 | 1985 | 1986 | 1987 | 1988 | 1989 | 1990 |
|--------|------|------|------|------|------|------|------|------|------|------|
| Number | 7    | 2    | 3    | 5    | 7    | 4    | 3    | 5    | 7    | 8    |

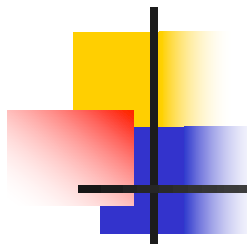
- Display the data with a time series graph.

# Time series graph – Example (cont.)



➤ Graph seem to display an upward trend over the years.

➤ Highest number was in 1990.

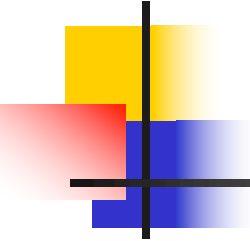


# Pareto Charts

---

- **What is a Pareto chart?** A Pareto chart is a type of bar chart in which the horizontal axis represents the categories of interest. The bars are ordered from largest to smallest in terms of the frequency counts for the categories.
- **Note:** A Pareto chart can help you determine which of the categories make up the critical few and which are the insignificant many.



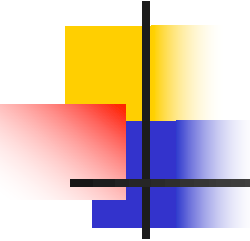


# Pareto Charts - Example

---

- A cumulative percentage line helps you judge the added contribution of each category.
- **Example:** The following information over a six month period, relating to the number of defects in a manufacturing process in a company, was obtained by the quality control team.
- The data is given on the next slide.
- Present a Pareto chart for the data.

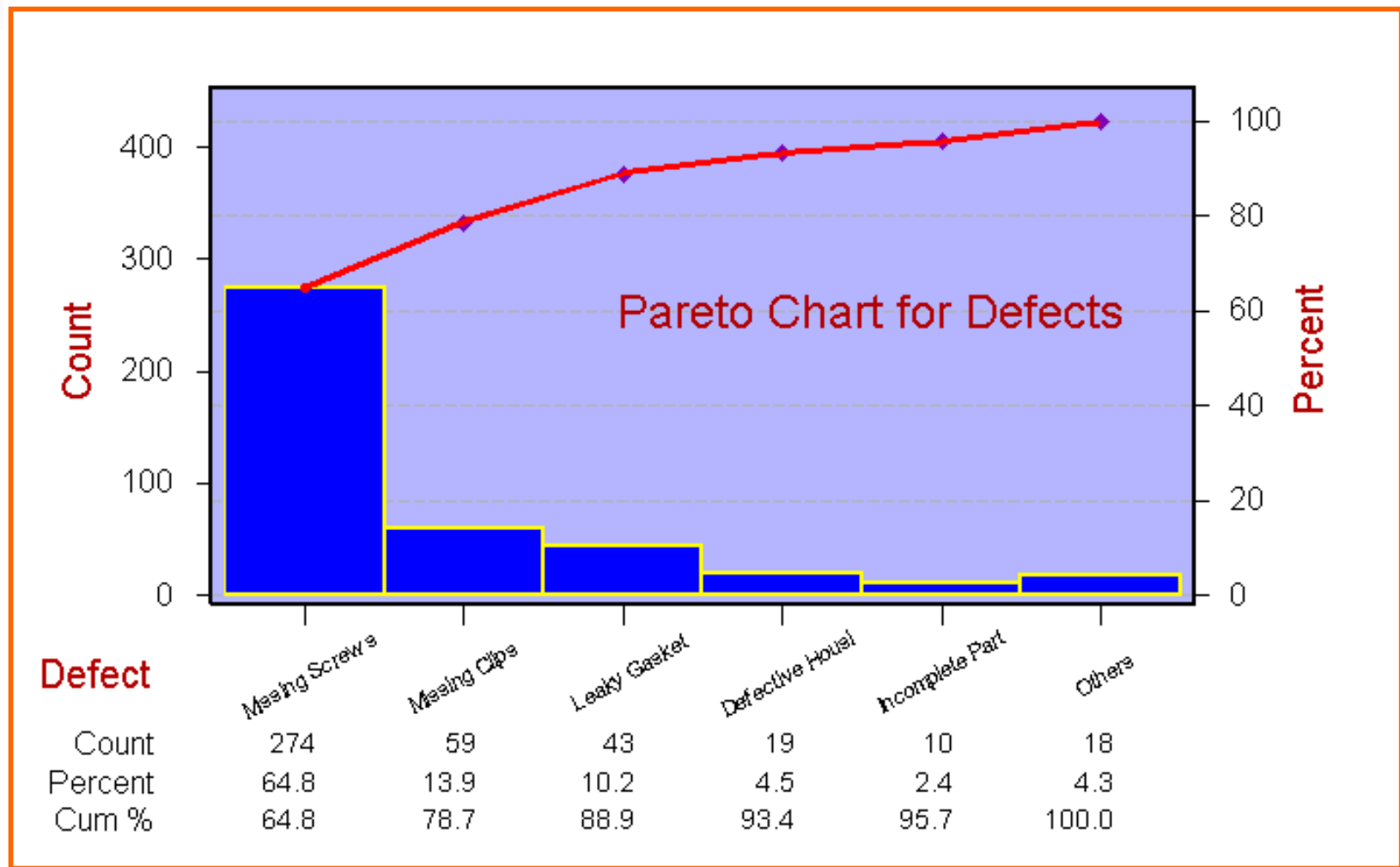
# Pareto Charts – Example (cont.)



| Defects           | Counts |
|-------------------|--------|
| Missing Screws    | 274    |
| Missing Clips     | 59     |
| Defective Housing | 19     |
| Leaky Gasket      | 43     |
| Scrap             | 4      |
| Unconnected Wire  | 8      |
| Missing Studs     | 6      |
| Incomplete Part   | 10     |

Note: The Pareto chart, on the next slide, combined the information for the scrap, unconnected wire, and missing studs as an Others category.

# Pareto Charts – Example (cont.)

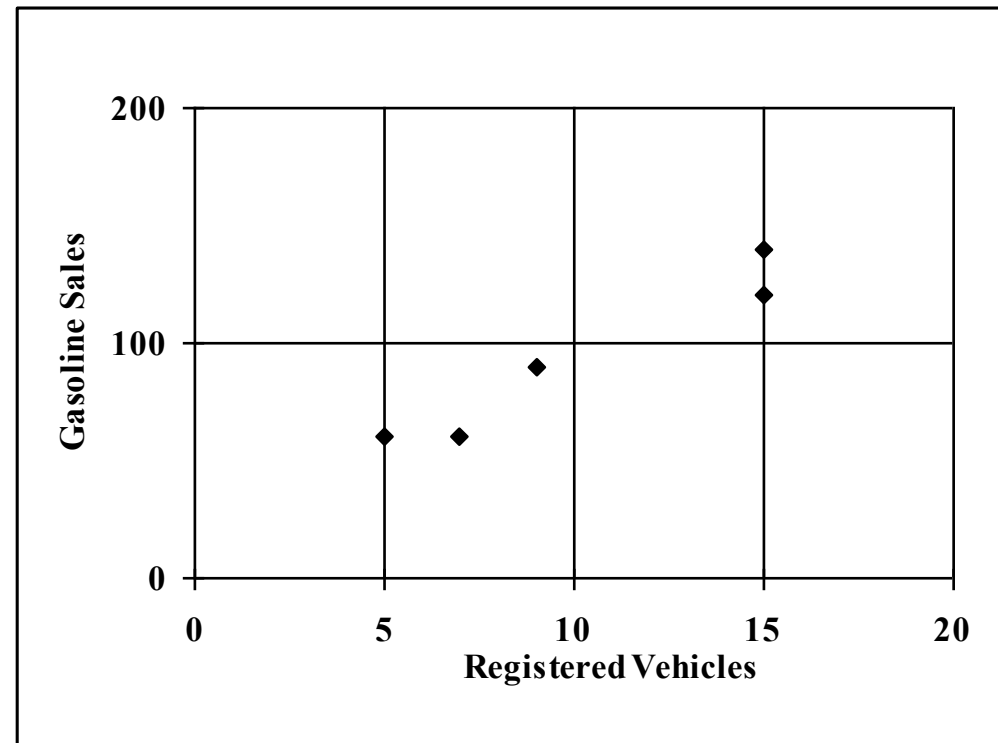


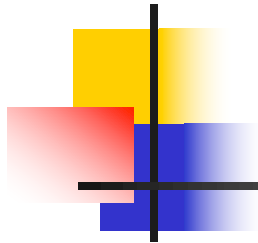
# Scatter Plot

**Registered  
Vehicles  
(1000's)**

**Gasoline Sales  
(1000's of  
Gallons)**

|    |     |
|----|-----|
| 5  | 60  |
| 15 | 120 |
| 9  | 90  |
| 15 | 140 |
| 7  | 60  |

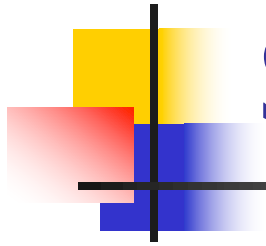




# Scatter Plot: R

---

- `x<-c(5,15,9,15,7)`
- `y<-c(60,120,90,140,60)`
- `plot(x, y, xlim=c(0,20), ylim=c(0,200),  
xlabel="Registered vehicles",  
ylabel="Gasoline sales")`



# Scatter Plot - Applications

---

- Assessing dependencies between variables
- Identify outliers (atypical values)
- Observe correlation

# Detecting Outliers

